IO多路复用--epoll和select

2020年11月21日 13:45

资料链接： https://www.cnblogs.com/fnlingnzb-learner/p/5835573.html
内核实现 https://www.baidu.com/link?url=6D9WG0-ivTqu97nwtH0SapcXI8Arznwlj_kr-JaaK4AoQfK3qKbo_J7L7oXybvj7TZ2u-bOHzl2ZSUT1vDEmEa&wd=&eqid=bf6a9d07002dbe6d00000006604e664a

注	fd：文件描述符号，类似文件句柄，file descriptor

epoll和select的区别	epoll和select都是用于多路复用的一些系统调用，可以监听多个文件描述符上的事件并进行处理。它们的主要区别在于： 1. 所处理的文件描述符数量不同：select使用数组来存储文件描述符，所以其能够处理的文件描述符的数量受到限制；而epoll使用红黑树来存储文件描述符，所以其能够处理的文件描述符数量不受限制。 2. 对于大量文件描述符的IO事件的处理效率不同：当需要同时处理大量的文件描述符时，epoll比select有更好的性能表现。这是因为在处理大量文件描述符时，epoll只需要遍历那些发生了事件的文件描述符，而select需要遍历所有的文件描述符。 3. select有一些平台特定的限制：在某些平台上，select可能不支持某些类型的文件描述符，而epoll没有这样的限制。 总之，尽管两者都可以实现多路复用，但在处理大量文件描述符时，epoll比select具有更好的性能和可扩展性，所以epoll更加常用 来自 <https://chat.theb.ai/#/chat/1002>
epoll和poll的区别	epoll是Linux内核提供的一种事件通知机制，可以用来实现I/O多路复用。相比于poll，epoll具有更高的性能和可伸缩性。在处理大量连接的情况下，epoll可以极大地减少系统的开销。 epoll的主要优点是： 1. 没有最大文件描述符数量的限制，可以支持上万个连接； 2. 支持水平触发和边缘触发两种工作模式； 3. 将事件存储在内核空间中，避免了每次调用都需要将事件从用户空间拷贝到内核空间，提高了I/O效率； 4. 可以使用mmap加速大规模数据传输。 而poll则没有这些优势，它的最大限制是1024个文件描述符，同时poll每次调用都需要将事件从用户空间拷贝到内核空间，所以效率上会差一些。 总的来说，如果你需要处理大量连接，那么使用epoll会更好一些。但如果你的连接数不是很多，那么使用poll也是可以的。 来自 <https://chat.theb.ai/#/chat/1002>
epoll为什么可以只遍历发生了事件的文件描述符（select和poll都只能轮询所有的）	epoll通过使用**内核支持的事件机制**，将所有需要监控的文件描述符存储在一个红黑树中，并使用一个链表来保存发生事件的文件描述符。这样，在检查是否有事件发生时，只需要遍历链表即可，而不用遍历整个红黑树。 当一个文件描述符上发生了事件时，内核会把这个文件描述符加入到一个链表中，然后唤醒等待该事件的进程。进程得到通知后，可以直接从链表中取出发生事件的文件描述符。 这种机制使得epoll在处理大量文件描述符时，性能更高。因为它不需要像select、poll那样每次

	都需要遍历整个文件描述符集，只需要遍历发生事件的文件描述符集合，就能够快速地找到并处理发生的事件。 另外，epoll还支持水平触发和边缘触发两种工作模式。在水平触发模式下，只要文件描述符上有数据可读或可写，就会返回事件通知；而在边缘触发模式下，只有在文件描述符状态发生变化时才返回事件通知。这种机制也能够提高epoll的性能和效率。

在Linux的网络编程中，很长的时间都在使用select做事件触发。在新的Linux内核中，有新的机制--epoll

一、select

1. 采用轮询处理fd，当fd数目多时，耗时越多
2. select同时监听的数据默认被宏定义为1024.虽然可以重新编译内核以扩大这个数，但治标不治本

二、epoll

epoll的接口非常简单，一共就三个函数：

1. int epoll_create(int size); //参数size为int意味着其支持最大的fd数目为2^31-1

创建一个epoll的句柄，size用来告诉内核这个监听的数目一共有多大。这个参数不同于select()中的第一个参数，给出最大监听的fd+1的值。需要注意的是，当创建好epoll句柄后，它就是会占用一个fd值，在linux下如果查看/proc/进程id/fd/，是能够看到这个fd的，所以在使用完epoll后，必须调用close()关闭，否则可能导致fd被耗尽。

2. int epoll_ctl(int epfd, int op, int fd, struct epoll_event *event);

epoll的事件注册函数，它不同与select()是在监听事件时告诉内核要监听什么类型的事件，而是在这里先注册要监听的事件类型。第一个参数是epoll_create()的返回值，第二个参数表示动作，用三个宏来表示：

EPOLL_CTL_ADD：注册新的fd到epfd中；

EPOLL_CTL_MOD：修改已经注册的fd的监听事件；

EPOLL_CTL_DEL：从epfd中删除一个fd；

第三个参数是需要监听的fd，第四个参数是告诉内核需要监听什么事，struct epoll_event结构如下：

```
typedef union epoll_data {
    void *ptr;
    int fd;
    __uint32_t u32;
    __uint64_t u64;
} epoll_data_t;

struct epoll_event {
    __uint32_t events; /* Epoll events */
    epoll_data_t data; /* User data variable */
};
```

events可以是以下几个宏的集合：

EPOLLIN ：表示对应的文件描述符可以读（包括对端SOCKET正常关闭）；

EPOLLOUT：表示对应的文件描述符可以写；
EPOLLPRI：表示对应的文件描述符有紧急的数据可读（这里应该表示有带外数据到来）；
EPOLLERR：表示对应的文件描述符发生错误；
EPOLLHUP：表示对应的文件描述符被挂断；
EPOLLET： 将EPOLL设为边缘触发(Edge Triggered)模式，这是相对于水平触发(Level Triggered)来说的。
EPOLLONESHOT：只监听一次事件，当监听完这次事件之后，如果还需要继续监听这个socket的话，需要再次把这个socket加入到EPOLL队列里

3. int epoll_wait(int epfd, struct epoll_event * events, int maxevents, int timeout);
等待事件的产生，类似于select()调用。参数events用来从内核得到事件的集合，maxevents告之内核这个events有多大，这个 maxevents的值不能大于创建epoll_create()时的size，参数timeout是超时时间（毫秒，0会立即返回，-1将不确定，也有说法说是永久阻塞）。该函数返回需要处理的事件数目，如返回0表示已超时。
4、关于ET、LT两种工作模式：
可以得出这样的结论:
ET模式仅当状态发生变化的时候才获得通知,这里所谓的状态的变化并不包括缓冲区中还有未处理的数据,也就是说,如果要采用ET模式,需要一直read/write直到出错为止,很多人反映为什么采用ET模式只接收了一部分数据就再也得不到通知了,大多因为这样;而LT模式是只要有数据没有处理就会一直通知下去的.

那么究竟如何来使用epoll呢？其实非常简单。
通过在包含一个头文件#include <sys/epoll.h> 以及几个简单的API将可以大大的提高你的网络服务器的支持人数。

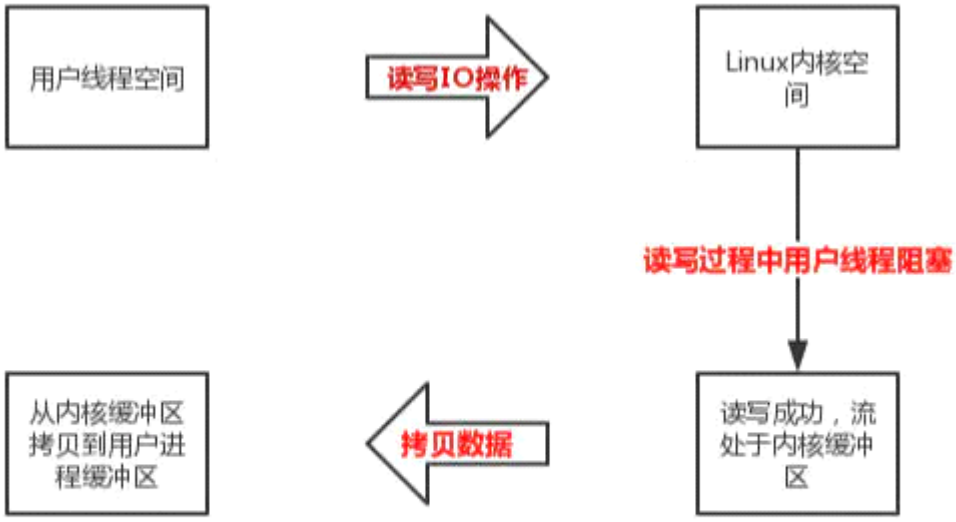
首先通过create_epoll(int maxfds)来创建一个epoll的句柄，其中maxfds为你epoll所支持的最大句柄数。这个函数会返回一个新的epoll句柄，之后的所有操作将通过这个句柄来进行操作。在用完之后，记得用close()来关闭这个创建出来的epoll句柄。

之后在你的网络主循环里面，每一帧的调用epoll_wait(int epfd, epoll_event events, int max events, int timeout)来查询所有的网络接口，看哪一个可以读，哪一个可以写了。基本的语法为：
nfds = epoll_wait(kdpfd, events, maxevents, -1);
其中kdpfd为用epoll_create创建之后的句柄，events是一个epoll_event*的指针，当epoll_wait这个函数操作成功之后，epoll_events里面将储存所有的读写事件。max_events是当前需要监听的所有socket句柄数。最后一个timeout是 epoll_wait的超时，为0的时候表示马上返回，为-1的时候表示一直等下去，直到有事件范围，为任意正整数的时候表示等这么长的时间，如果一直没有事件，则范围。一般如果网络主循环是单独的线程的话，可以用-1来等，这样可以保证一些效率，如果是和主逻辑在同一个线程的话，则可以用0来保证主循环的效率。

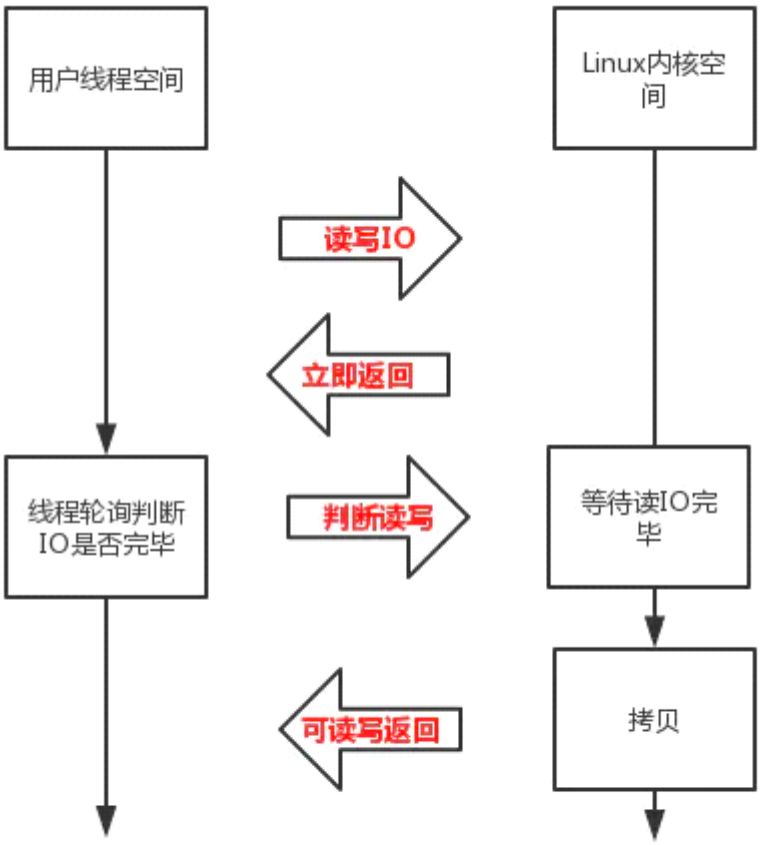
epoll_wait范围之后应该是一个循环，遍利所有的事件。

IO多路复用

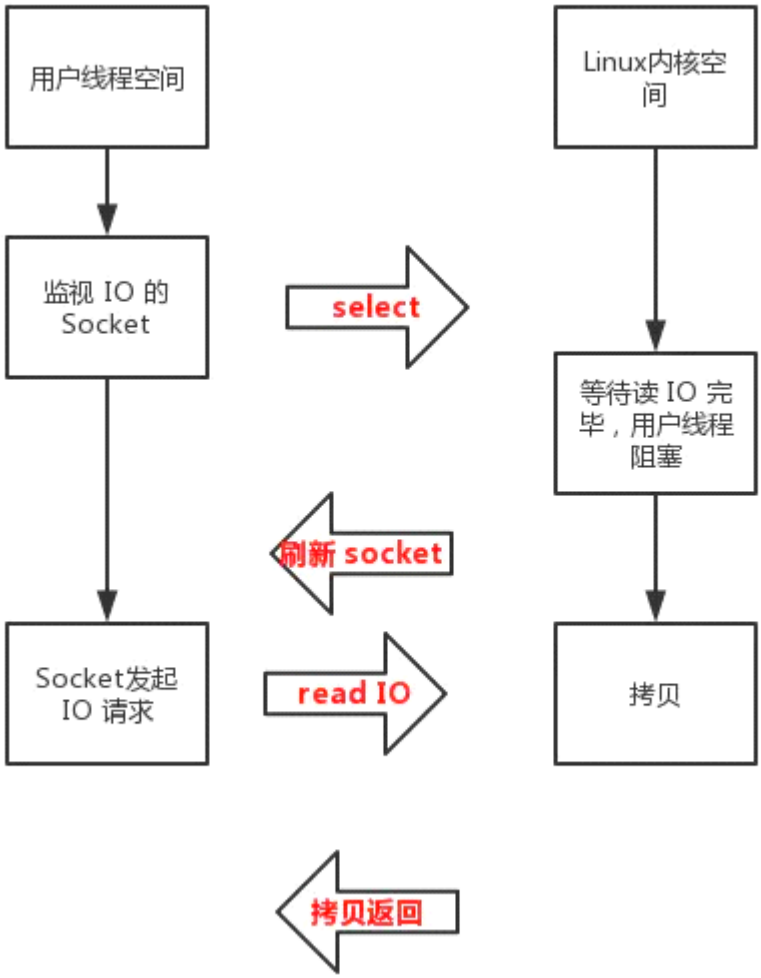
2021年3月15日 3:43



同步阻塞 IO 模型



同步非阻塞IO模型



IO 多路复用的核心就在于同一时刻一个逻辑流也就是一个线程可以监听操作多个 IO，而其他 IO 模型只能通过多线程来进行多个 IO 的需求。

来自 <https://www.jianshu.com/p/9d46d82803dd>

关于main函数与进程、线程退出的关系

2021年3月19日 3:01

关于main函数与进程、线程退出的关系
在学习嵌入式系统的进程和线程时，有以下的疑问：
讲师说进程退出时，进程中的所有线程都会退出；但是主线程退出时，子线程是不会退出的。
当时觉得主线程=主进程main函数，那主线程退出了，子线程还会继续执行呢？

在网上查阅资料之后，弄明白了三者的关系就理解了开头的两句话，下面写一下总结，方便以后复习。

理解进程与线程
进程 是系统分配资源的最小单位。
线程 是系统调度的最小单位。

线程是程序最基本的运行单位，而进程是不能运行的，运行的是进程中的线程。创建进程时，系统会自动创建一个主线程来运行进程中的数据信息。

我们运行main函数的时候，就会创建一个主进程和一个主线程，当我们在主线程中创建更多的子线程后：
调用exit()函数：主进程和主线程都退出，子线程自然也要退出。
调用pthread_exit()函数：主线程退出，但是主进程依旧存在，所以进程中的子线程可以继续运行。

在程序中使用多进程多线程时，曾经出现ctrl c强制退出程序后，仍然有线程在后台执行，要手动kill掉。是因为父进程死亡后，主进程中的线程跟随主进程一起退出了，而子进程由系统1号进程进行管理，所以子进程中的线程还可以继续运行。这就导致我在调试运行程序的时候经常会出现报错，原因是因为后台中的子进程还在运行导致的。所以在处理进程和线程时，要几时对线程和进程资源进行回收，避免出错。

版权声明：本文为CSDN博主「jarvis_zz」的原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接及本声明。
原文链接：https://blog.csdn.net/weixin_43352468/article/details/107127444

关于Linux根“/”的思考

2023年1月18日 10:07

注意，文件系统这个是对于磁盘等外部设备来说的，因为这个文件系统就是用来管理外部设备。

对于一个计算机来说，即使没有磁盘（外部设备）也是能运行起来的（只要把所有需要的代码、数据放在内存上（如虚拟机）。

所以为什么说“Unix创建了一个根目录，并包含在根文件系统中”，也就是即使，没有这个根文件系统，Linux还是Linux（一个能运行的大型程序），但显然这个根文件系统是不可或缺的。另一个侧面解释了之前思考“根“/”从何而来？，为什么能被别的磁盘挂载，这个磁盘为什么就已经存在“根”，还挂载到自己的目录上“，因为”Linux“这个大型程序已经设计好这个程序了！实际这个磁盘不是挂载到自己身上，而是挂在了在内存上的根；也就是设计好这个根文件系统了。

Linux内核三种错误

2023年1月18日 10:08

1. BUG
2. Oops
3.Panic

Linux各种类型的进程

2023年1月18日 10:08

1. 僵死（尸）进程：一个进程使用fork创建子进程，如果子进程退出，而父进程并没有调用wait或waitpid获取子进程的状态信息，那么子进程的进程描述符仍然保存在系统中。这种进程称之为僵死进程。
2. 孤儿进程：一个父进程退出，而它的一个或多个子进程还在运行，那么那些子进程将成为孤儿进程。孤儿进程将被init进程(进程号为1)所收养，并由init进程对它们完成状态收集工作。
3. 守护进程
4. 协同进程

终端的用户与主机名

2023年1月18日 10:13

首先这个属于Linux中的$PS1的变量,你可以通过 echo $PS1查看你当前的变量值.
默认的PS1变量值是 [\u@\h \W]\$,每个参数的含义
"root":表示当前的登录的用户
"@":可以理解为是一个分隔符号
"xxx-desktop":应该是你的主机名
"~":表示你当前所在的目录
"#":表示你当前登录用户的身份是root超级用户

来自 <https://www.lmlphp.com/user/57828/article/item/2442296/>

Linux五个子系统

2023年1月18日 10:13

1. 进程（调度）管理
2. 内存管理
3. 虚拟文件系统
4. 网络接口
5. 进程间通信

用户空间	应用程序 GNU C库
内核空间	系统调用接口 内核 架构依赖性内核代码 ---------- 硬件平台

睡眠、阻塞、挂起的区别

2023年2月17日 17:06

进程挂起，阻塞和睡眠的区别 - 知乎 (zhihu.com)

注	从下面描述看，三者都会占用调度器。

它们表现形式很相似：正在执行的进程/线程，由于某些原因(主、被动)释放CPU，暂停执行。

从触发机制上：

睡眠是主动触发的，

挂起也是主动触发

而阻塞是被动的

从资源占用角度来说：

阻塞进程不再占用CPU资源，但还在占用调度器和内存资源

挂起进程不再占用CPU和内存资源了

睡眠进程既占用内存资源又占用CPU资源

补充：

任务调度是操作系统来实现的，任务调度时，直接忽略挂起状态的任务，但是会顾及处于pend下的任务，当pend下的任务等待的资源就绪后，就可以转为ready了。ready只需要等待CPU时间，当然，任务调度也占用开销，但是不大，可以忽略。可以这样理解，只要是挂起状态，操作系统就不再管理这个任务了。

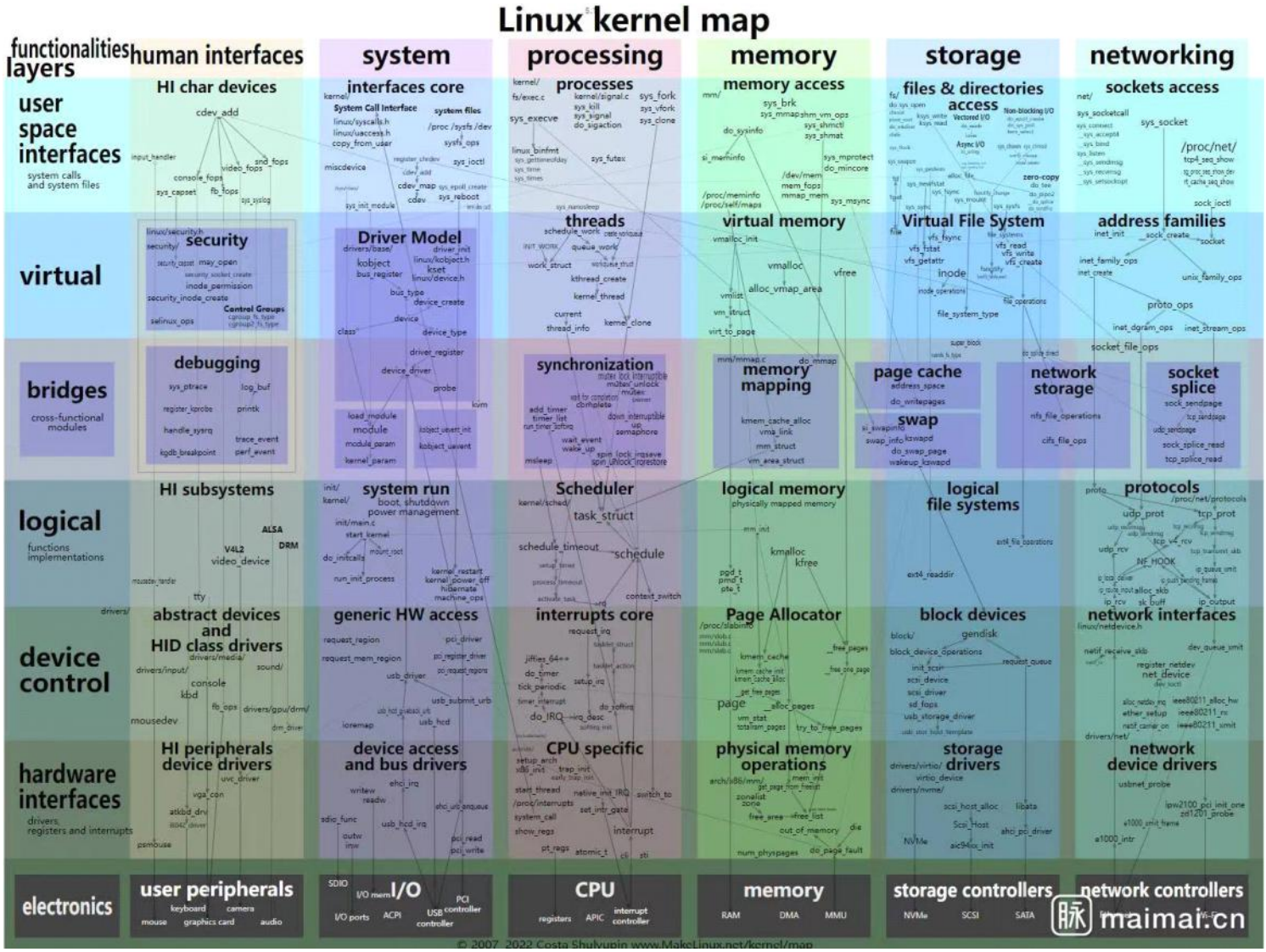
从恢复上来看：
睡眠恢复是自动完成的，因为睡眠有一个睡眠时间，睡眠时间到则恢复到就绪态。而阻塞是被动的，是在等待某种事件或者资源的表现，一旦获得所需资源或者事件信息就自动回到就绪态。最后挂起情况最多，如果是用户将进程挂起的话，那么只有用户才能解除进程的挂起状态，而如果是被操作系统挂起的，这要根据系统资源情况，优先级，挂起中的状态来判定什么时候解除挂起状态。

拜读了其他博主的文章，找到一个很好的理解挂起，阻塞，睡眠三者区别的例子，我这里引用一下：

挂起线程的意思就是你对主动对雇工说：“你睡觉去吧，用着你的时候我主动去叫你，然后接着干活”。
使线程睡眠的意思就是你主动对雇工说：“你睡觉去吧，某时某刻过来报到，然后接着干活”。
线程阻塞的意思就是，你突然发现，你的雇工不知道在什么时候没经过你允许，自己睡觉呢，但是你不能怪雇工，肯定你这个雇主没注意，本来你让雇工扫地，结果扫帚被偷了或被邻居家借去了，你又没让雇工继续干别的活，他就只好睡觉了。至于扫帚回来后，雇工会不会知道，会不会继续干活，你不用担心，雇工一旦发现扫帚回来了，他就会自己去干活的。因为雇工受过良好的培训。这个培训机构就是操作系统。

内核总览图

2023年3月1日 9:26



文件与目录所有者、所属组、UID、GID、用户名

2023年3月29日 17:14

在Linux系统中，每个文件和目录都有一个所有者（owner）和一个所属组（group）。这个数字1002表示该文件或目录的所有者的用户ID（UID），也就是该用户在系统中的唯一标识符。
通常情况下，Linux系统会将每个新创建的用户分配一个唯一的UID，以便能够正确地跟踪和管理文件和目录的访问权限。因此，如果您看到一个文件的所有者是1002，那么可以通过查找/etc/passwd文件来确定该UID标识的实际用户名。

2023/3/29 17:12:35
/etc/passwd文件内容含义

2023/3/29 17:13:00
/etc/passwd文件是一个文本文件，包含了系统上每个用户的基本信息。该文件的每一行表示一个用户，其中包括以下七个字段：

1. 用户名：用于登录系统的用户名
2. 密码：加密后的密码（现在通常存储在/etc/shadow文件中）
3. 用户ID（UID）：每个用户在系统中唯一的标识符
4. 组ID（GID）：每个用户所属的主要组的标识符
5. 用户信息（注释）：任意的用户说明信息
6. 主目录：用户登录时的默认目录
7. 登录 shell：用户登录时使用的shell程序路径

例如，下面是一个/etc/passwd文件的样例行：

复制代码
```
john:x:1000:1000:JohnSmith:/home/john:/bin/bash
```

这表示有一个名为“john”的用户，他的UID和GID都是1000，他的注释是“John Smith”，他的主目录是/home/john，他的登录shell是/bin/bash。注意到密码字段中包含了一个"x"字符，这意味着密码已经被移动到了/etc/shadow文件中。

来自 <https://www.aiai.zone/#/chat/1680058804112>

dmesg和/var/log/messages

2023年4月19日 14:46

2023/4/19 14:45:28
dmesg解析的是哪个文件

2023/4/19 14:45:32
dmesg 命令解析的是 Linux 内核环缀日志，其源数据来源于内核缓冲区，而非文件。

2023/4/19 14:45:53
/var/log/messages包含什么内容

2023/4/19 14:46:01
/var/log/messages 是一个 Linux 系统日志文件，其中包含了操作系统和应用程序的大量信息记录，例如系统启动、关机、内核消息、网络服务、安全事件及其他进程产生的消息等。具体包含的内容因不同的系统配置和使用情况而异，通常是一个综合性的日志文件，可用于排查问题和对系统进行监控。

来自 <https://mirrorchat.extkj.cn/#/chat/1681886702802>

未命名

2023年6月2日 15:47

sg链表（scatter-gather链表）是一种链表结构，用于在Linux内核中有效处理IO操作。它的作用是可以在支持scatter/gather IO复制的基本设备（例如DMA引擎）之间传递数据。sg链表可以被看作是逻辑上分散或聚集的数据块的表示。在处理IO操作时，这些数据块可以是在不同物理地址上的不连续块，也可以是在一个缓冲区中的连续块。

sg链表的结构如下所示：

```
struct scatterlist {
   unsigned long    page_link;
   unsigned int    length;
   dma_addr_t      dma_address;
};
```

它是由多个scatterlist结构体以链表形式连接而成的。每个scatterlist结构体表示一个页或一个虚拟内存地址范围。chain中独立的每个scatterlist结构体代表了一个数据块，中间可以有一些空洞。

scatterlist结构体中的主要字段包括：

- page_link：一个指针，指向下一个scatterlist结构体的地址。
- length：所代表的数据块的长度。
- dma_address：将虚拟地址转换为物理地址后，代表数据块在物理地址上的起始位置。

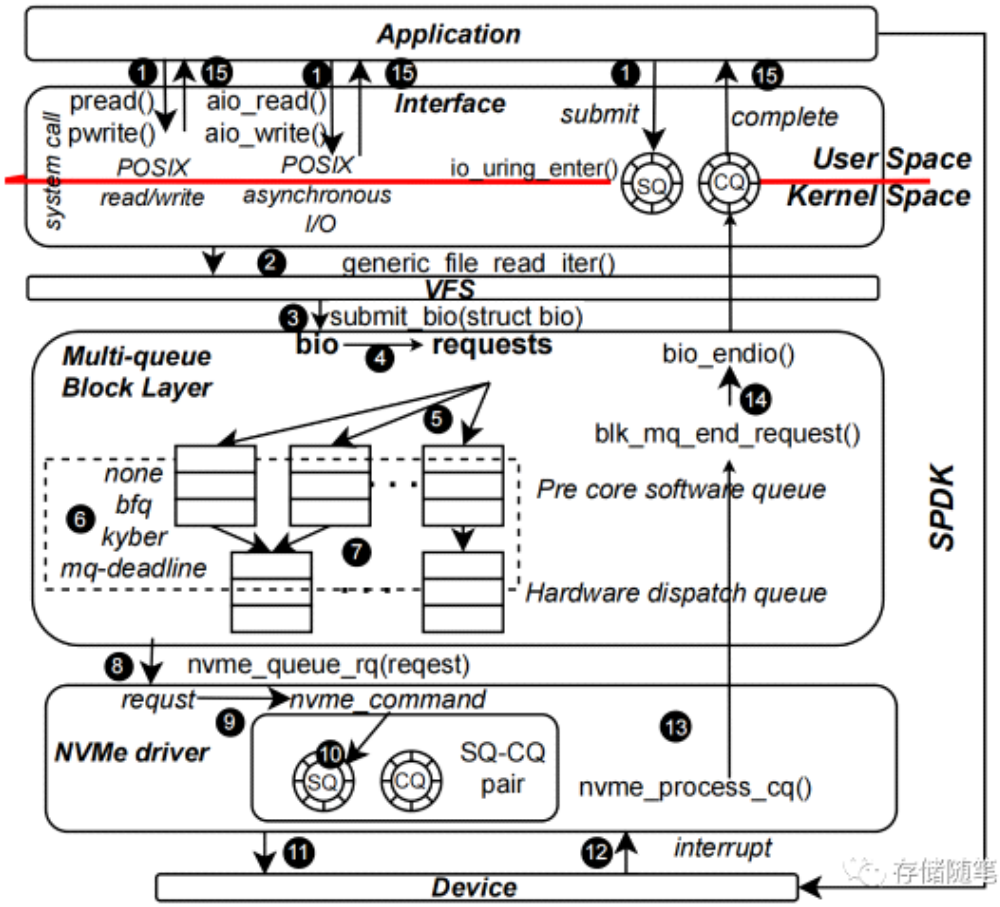
sg链表主要应用于块设备和网络驱动程序的数据传输。使用sg链表来传输数据时，每个scatterlist结构体代表一个数据块，并相互链接成一个链，这个链就是sg链表。在处理IO数据时，内核通过遍历sg链表来实现数据复制。

四种IO栈

2023年9月14日 22:55

1.Linux IO栈处理

我们就从一个常见Linux IO处理流程示意图开始今天的主题。



这个示意图主要描述了Linux系统中I/O请求的处理流程，涉及了I/O接口、文件系统、块层、NVMe驱动等多个部分，整个IO流程处理主要概括以下步骤：

1. 应用程序（Applications）将I/O请求提交给I/O接口（Step 1）。这些请求可以是POSIX的read/write操作（如psync）、libaio、io_uring等。

2. I/O接口构建一个struct kiocb结构体，并通过VFS（虚拟文件系统）将其提交给文件系统（Step 2）。

3. 文件系统解析包含数据的块地址，构建一个struct bio请求，并通过submit_bio()函数将其提交给块层（Step 3）。

4. Linux块层将struct bio转换为struct request（Step 4），并将其放入每个核心的软件队列（Step 5）中。

5. 然后，这些请求由块I/O调度器处理，并放入硬件调度队列（Step 6-7）中。

6. 硬件调度队列中的请求将被NVMe驱动程序通过nvme_queue_rq()函数处理（Step 8）。

7. NVMe驱动程序根据请求构建一个NVMe命令（Step 9），并将其写入提交队列（SQ）（Step 10）以供设备进行处理（Step 11）。

8. 在设备处理完命令后，设备将请求写入完成队列（CQ）并生成中断（Step 12）。

9. 完成队列中的请求由nvme_process_cq函数处理，然后调用blk_mq_end_request()和bio_end_io()函数来完成bio层的请求（Step 14）。

10. 最后，应用程序将收到完成结果的通知（Step 15）。

2.不同种类IO栈解读

在上述IO处理流程第一步中，IO接口种类有四种：

- **POSIX I/O**

是在Linux内核版本2.6中引入。POSIX I/O，全称Portable Operating System Interface，是一种基于同步阻塞的系统调用接口，提供标准的输入输出接口，例如open、read、write、close等函数。在POSIX I/O中，应用程序发起I/O请求，然后阻塞当前进程，等待I/O操作完成。在这个过程中，如果发生错误或者其他事件，应用程序可以通过errorfd等机制获取到相关信息。

这可是I/O界的元老。它的原理其实很简单：当你要读取或者写入数据时，它会使用标准的系统调用来完成这个任务。这就好像你去餐厅点餐，服务员直接把菜单拿到厨房一样。

- **libaio**

在Linux内核版本2.6.32中引入的。libaio是一种异步I/O库，提供异步I/O操作接口，它发起I/O请求后，不会阻塞当前进程，而是立即返回，让应用程序可以继续执行其他任务。当I/O操作完成后，libaio会通过回调函数通知应用程序。这种方式可以提高应用程序的并发性能和响应速度。Libaio的优点在于其使用简单，适用于需要大量并发I/O操作的应用程序。

它的原理和POSIX I/O不同。它使用的是一种叫做"发起并忘记"的模式，意思就是当你告诉libaio你要读取或者写入数据后，它就会自己去完成这个任务，而不需要你一直等待。这就像你点餐后，服务员告诉你饭菜已经准备好了，然后你自己去取一样。

- **SPDK**

是一种基于用户空间的高性能数据存储解决方案，它绕过了Linux内核来实现数据传输。这意味着SPDK可以在不依赖内核的情况下进行数据传输，从而提高了性能和效率。

具体来说，SPDK通过使用用户空间进程、虚拟文件系统和内存映射等技术来处理数据I/O操作，而不是直接与Linux内核交互。这使得SPDK可以更灵活地处理不同的硬件和操作系统环境，并且可以减少对内核的依赖性。此外，由于绕过了内核，SPDK还可以获得更好的安全性和可靠性，因为内核级别的漏洞和错误不会直接影响到用户空间应用程序。

它的原理是利用一种叫做"Polling AIO"的技术，这种技术可以在没有回调的情况下发起并处理I/O操作。这就像餐厅里的机器人服务员，不仅快速而且高效！

- **io_uring**

在Linux内核版本5.1中引入的。io_uring是Linux内核中的一种新型I/O接口，它只有一种工作模式：异步I/O（async IO）。这种接口允许应用程序在发起I/O请求后不阻塞当前进程，而是立即返回，让应用程序可以继续执行其他任务。当I/O操作完成后，io_uring会通过回调函数通知应用程序。

io_uring也可以提交轮询模式的I/O请求，这种模式的工作原理是在用户空间中实现一个零中断、零复制、轮询驱动的NVMe驱动程序，将PCIe寄存器映射到用户空间，用于配置设备和应用程序之间共享的SQs（submission）和CQs（completion）队列。轮询模式可以提高I/O操作的并发性能和响应速度，但是需要文件系统（如果有）和块设备（block device）支持轮询功能。

io_uring还可以通过io_uring_enter()函数提交I/O请求，然后直接检查CQ状态判断是否完成。相比传统的同步I/O模型，io_uring可以更好地利用多核处理器和并行计算的优势，提高I/O操作的并发性能。

需要注意的是，io_uring不支持混合使用轮询和非轮询模式。

它的原理有点类似于SPDK的"Polling AIO"，但是它更加通用，可以支持多种I/O操作，包括随机读、顺序读、随机写、顺序写等。它就像一个多功能的自助餐厅，可以满足各种不同的口味需求！

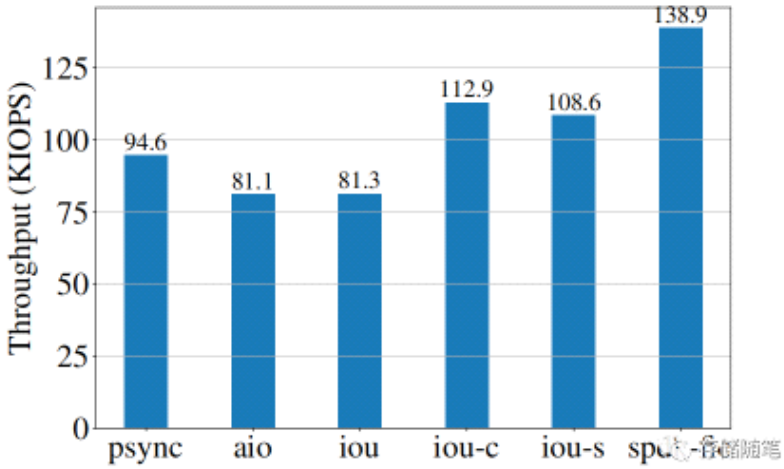
总的来说，这四种I/O技术各有特点：

- **POSIX I/O就像一个老实巴交的餐厅服务员，稳稳当当；**
- **libaio就像是个性化的外卖服务，便捷高效；**
- **SPDK就像个神秘的厨师，用心做出美味的佳肴；**
- **io_uring就像是个全能的大厨，能做出各种美食而且还能根据你的喜好调整口味！**

3.不同IO栈性能差异对比

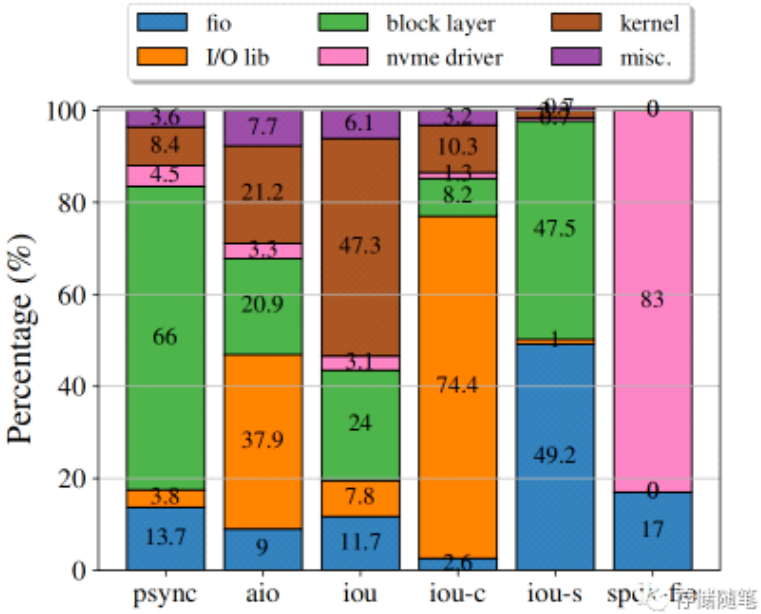
单QD队列4K随机读场景下：

- 六种IO模式下，aio/iou表现最差。
- iou三种对比，基于轮询模式的iou-s和iou-c比非轮训模式iou要高出34%、39%的IOPS性能。
- SPDK表现最好。



在执行IO过程中，在不同的IO模型下，IO开销在不同阶段占比是不一样的。

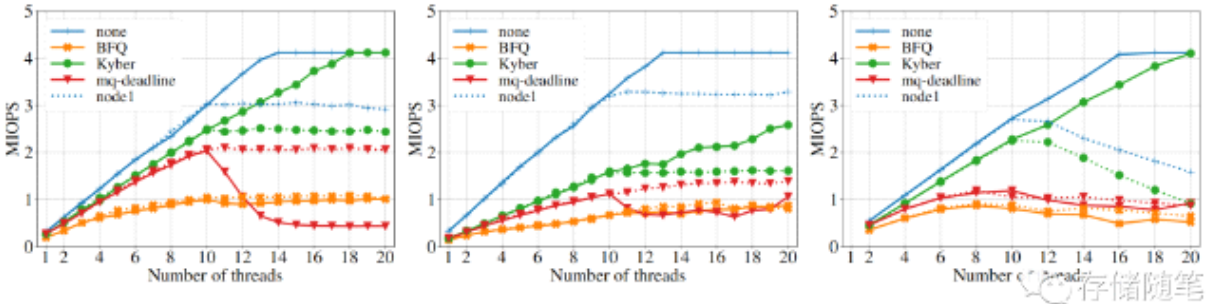
- *在理想情况下，所有CPU指令（100%）都将被fio消耗，从而显示零成本的理想的I/O API，然而，这样的系统是不现实的*
- psync在块设备层开销时间最大，占比66%。这也是大家为何摒弃它的原因，内核耗时太长了。
- libaio在块设备层开销不算多，但是在自身IO实现中开销占比近40%。
- 默认的iou主要开销在内核，但随着轮询的引入，iou-c将其大部分指令用于完成轮询
- SPDK 83%的开销在NVME驱动的IO轮询上。



基于io_uring模式下，不同IO调度器的影响也有很大的不同。如之前文章中介绍，

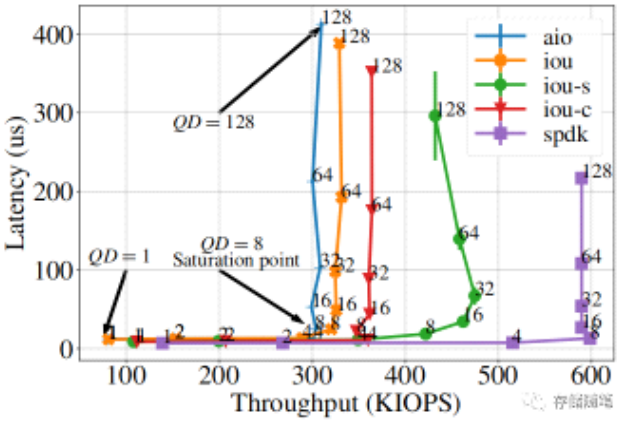
http://mp.weixin.qq.com/s?__biz=MzIwNTUxNDgwNg==&mid=2247486596&idx=1&sn=2d88a6789de9500071c8c14e8b53c146&chksm=972efddda05974cbf2a74914baf6b8425a640466dd9c4d187d5037e553eb24eb2b0987b2bd8c&scene=21#wechat_redirect

Noop调度器是最适用于固态硬盘（SSD）等存储设备，因为SSD IO调度算法越简单，效率就越高。



(a) default io_uring (iou). (b) with completion polling (iou-c). (c) with submission polling (iou-s).

在不同的IO队列深度vs带宽对比中，SPDK可以实现高性能低延迟的特性。



FHS（文件系统层次化标准）

2023年10月30日 9:20

两层规范：

1. ‘/’下面的各个目录应该要放什么文件数据
2. 针对/usr及/var这两个目录的子目录来定义，其应该放什么文件数据

四种交互作用的形态：

1. 可分享的：可以分享给其他系统挂载使用的目录
2. 不可分享的：自己机器上运作的装置文件或者是与程序有关的socket文件等，由于仅与自身机器有关，所以不适合分享给其他主机
3. 不变的：有些数据是不会经常变动的，例如函数库、文件说明文件、系统管理员所管理的主机服务器配置文件等
4. 可变动的：经常改变的数据

<table>
<tr><th>目录</th><th>解释</th></tr>
<tr><td>/</td><td>根</td></tr>
<tr><td>/bin</td><td>存放所有用户都可以使用的Linux基本操作命令</td></tr>
<tr><td>/boot</td><td>存放开机启动加载程序的核心文件</td></tr>
<tr><td>/dev</td><td>设备文件目录，虚拟文件系统</td></tr>
<tr><td>/home</td><td>默认存放用户的主目录（除了root用户）</td></tr>
<tr><td>/lib</td><td>包含许多由/bin和/sbin中的程序使用的共享库文件</td></tr>
<tr><td>/lost+found</td><td>一般情况下为空。文件系统修复</td></tr>
<tr><td>/misc</td><td>自动挂载目录，对应autofs服务</td></tr>
<tr><td>/proc</td><td>虚拟文件系统，此目录是kernel加载后，在内存里面建立的一个虚拟目录，有专属的文件系统，主要提供系统一些实时的信息，此目录下不能建立和删除文件（某些文件可以修改）。
作用：1.整理系统内部的信息；2.存放主机硬件信息；3.调整系统执行时的参数；4.检查及修改网络和主机的参数；5.检查及调整系统的内存和性能
<table>
<tr><td>/proc/cpuinfo</td><td>CPU的硬件信息</td></tr>
<tr><td>/proc/devices</td><td></td></tr>
<tr><td>/proc/filesystems</td><td></td></tr>
<tr><td>/proc/loadavg</td><td></td></tr>
<tr><td>/proc/ioports</td><td></td></tr>
<tr><td>/proc/kcore</td><td></td></tr>
</table></td></tr>
</table>

<table>
<tr><td></td><td><table>
<tr><td>/proc/kmsg</td><td></td></tr>
<tr><td>/proc/meminfo</td><td></td></tr>
<tr><td>/proc/modules</td><td></td></tr>
<tr><td>/proc/mtrr</td><td></td></tr>
<tr><td>/proc/iomem</td><td></td></tr>
<tr><td>/proc/partitions</td><td></td></tr>
<tr><td>/proc/数字</td><td></td></tr>
<tr><td>/proc/net</td><td></td></tr>
<tr><td>/proc/scsi</td><td></td></tr>
<tr><td>/proc/self</td><td></td></tr>
<tr><td>/proc/stat</td><td></td></tr>
<tr><td>/proc/uptime</td><td></td></tr>
<tr><td>/proc/version</td><td></td></tr>
</table></td></tr>
<tr><td>/sbin</td><td>系统管理相关的二进制文件存放在这个目录下，一些可执行文件普通用户具备较小的权限；(多数管理命令默认只有管理员可以使用)</td></tr>
<tr><td>/srv</td><td>默认为空，某些软件可能会把配置文件默认存放在这个目录</td></tr>
<tr><td>/usr</td><td>安装除操作系统本身外的一些应用程序或组件<table>
<tr><td>/usr/bin</td><td>一般用户有机会使用到的程序</td></tr>
<tr><td>/usr/sbin</td><td>一些系统有可能会用到的系统命令</td></tr>
<tr><td>/usr/etc</td><td>自行安装或非系统主要的配置文件目录</td></tr>
<tr><td>/usr/include</td><td></td></tr>
<tr><td>/usr/lib</td><td></td></tr>
<tr><td>/usr/local</td><td>Linux系统中安装的共享软件程序最好的方式是安装在本目录下</td></tr>
<tr><td>/usr/src</td><td></td></tr>
</table></td></tr>
<tr><td>/etc</td><td>主机、系统或网络配置文件存放目录</td></tr>
<tr><td>/media</td><td>移动存储设备默认挂载点（如光盘），自动挂载</td></tr>
<tr><td>/mnt</td><td>临时文件系统的挂载点</td></tr>
<tr><td>/opt</td><td>额外所安装的应用程序目录</td></tr>
<tr><td>/root</td><td>管理员root的宿主目录</td></tr>
<tr><td>/sys</td><td>虚拟文件系统；以分类的方式将系统的信息存放在这个目录</td></tr>
<tr><td>/tmp</td><td>临时文件存放区域（默认被配置了粘滞位）</td></tr>
<tr><td>/var</td><td>动态文件或数据存放目录</td></tr>
</table>

/sys/module/${modulename}/${paramter}

2023年1月4日 16:17

修改使用了module_param的内核全局变量可以通过修改文件方式修改其值。注意需要指定相应权限标志。

修改后，在在内核程序中也是同步修改的！不只是模块加载时传递。

内核测试接口速度

2023年3月22日 15:57

在 Linux 内核中测试一个接口的运行时间可以使用 `ktime_get()` 函数。该函数返回一个 `ktime_t` 类型的值，表示从系统启动到当前的纳秒数。

具体来说，您可以通过在接口的开始和结束处记录两个时间戳，然后计算它们之间的差值来得到该接口的运行时间。示例如下：

c复制代码

```c
#include<linux/ktime.h>// 测试接口，参数 arg 是接口的参数voidmy_interface(intarg){
    ktime_tstart_time, end_time;
    s64 delta_ns;

// 记录开始时间start_time = ktime_get();

// 调用接口// ...// 记录结束时间end_time = ktime_get();

// 计算时间差delta_ns = ktime_to_ns(ktime_sub(end_time, start_time));
    printk(KERN_INFO "my_interface took %lld ns to run\n", delta_ns);
}
```

在上面的代码中，`ktime_get()` 函数被用于获取当前时间。`ktime_sub()` 函数可以计算两个 `ktime_t` 类型值的差，而 `ktime_to_ns()` 函数将其转换为纳秒数。

请注意，在内核中测量接口的运行时间可能会影响系统的性能，因此应该尽量避免在生产环境中使用。此外，应该确保测试的准确性和可重复性，以便进行有意义的比较和优化。

来自 <http://newb.site/chatgpt/#/chat/1002>

命令行的艺术

2023年1月17日 9:21

前言

涵盖范围：

- 这篇文章不仅能帮助刚接触命令行的新手，而且对具有经验的人也大有裨益。本文致力于做到*覆盖面广*（涉及所有重要的内容），*具体*（给出具体的最常用的例子），以及*简洁*（避免冗余的内容，或是可以在其他地方轻松查到的细枝末节）。在特定应用场景下，本文的内容属于基本功或者能帮助您节约大量的时间。
- 本文主要为 Linux 所写，但在仅限 OS X 系统章节和仅限 Windows 系统章节中也包含有对应操作系统的内容。除去这两个章节外，其它的内容大部分均可在其他类 Unix 系统或 OS X，甚至 Cygwin 中得到应用。
- 本文主要关注于交互式 Bash，但也有很多技巧可以应用于其他 shell 和 Bash 脚本当中。
- 除去“标准的”Unix 命令，本文还包括了一些依赖于特定软件包的命令（前提是它们具有足够的价值）。

注意事项：

- 为了能在一页内展示尽量多的东西，一些具体的信息可以在引用的页面中找到。我们相信机智的你知道如何使用 Google 或者其他搜索引擎来查阅到更多的详细信息。文中部分命令需要您使用 apt-get，yum，dnf，pacman， pip 或 brew（以及其它合适的包管理器）来安装依赖的程序。
- 遇到问题的话，请尝试使用 Explainshell 去获取相关命令、参数、管道等内容的解释。

基础

- 学习 Bash 的基础知识。具体地，在命令行中输入 man bash 并至少全文浏览一遍；它理解起来很简单并且不冗长。其他的 shell 可能很好用，但 Bash 的功能已经足够强大并且到几乎总是可用的（ 如果你*只*学习 zsh，fish 或其他的 shell 的话，在你自己的设备上会显得很方便，但过度依赖这些功能会给您带来不便，例如当你需要在服务器上工作时）。

- 熟悉至少一个基于文本的编辑器。通常而言 Vim （vi） 会是你最好的选择，毕竟在终端中编辑文本时 Vim 是最好用的工具（甚至大部分情况下 Vim 要比 Emacs、大型 IDE 或是炫酷的编辑器更好用）。

- 学会如何使用 man 命令去阅读文档。学会使用 apropos 去查找文档。知道有些命令并不对应可执行文件，而是在 Bash 内置好的，此时可以使用 help 和 help -d 命令获取帮助信息。你可以用 type 命令 来判断这个命令到底是可执行文件、shell 内置命令还是别名。

- 学会使用 > 和 < 来重定向输出和输入，学会使用 | 来重定向管道。明白 > 会覆盖了输出文件而 >> 是在文件末添加。了解标准输出 stdout 和标准错误 stderr。

- 学会使用通配符 * （或许再算上 ? 和 [...]） 和引用以及引用中 ' 和 " 的区别（后文中有一些具体的例子）。
- 熟悉 Bash 中的任务管理工具：&，ctrl-z，ctrl-c，jobs，fg，bg，kill 等。
- 学会使用 ssh 进行远程命令行登录，最好知道如何使用 ssh-agent，ssh-add 等命令来实现基础的无密码认证登录。
- 学会基本的文件管理工具：ls 和 ls -l （了解 ls -l 中每一列代表的意义），less，head，tail 和 tail -f （甚至 less +F），ln 和 ln -s （了解硬链接与软链接的区别），chown，chmod，du （硬盘使用情况概述：du -hs *）。 关于文件系统的管理，学习 df，mount，fdisk，mkfs，lsblk。知道 inode 是什么（与 ls -i 和 df -i 等命令相关）。
- 学习基本的网络管理工具：ip 或 ifconfig，dig。
- 学习并使用一种版本控制管理系统，例如 git。
- 熟悉正则表达式，学会使用 grep / egrep，它们的参数中 -i，-o，-v，-A，-B 和 -C 这些是很常用并值得认真学习的。
- 学会使用 apt-get，yum，dnf 或 pacman （具体使用哪个取决于你使用的 Linux 发行版）来查找和安装软件包。并确保你的环境中有 pip 来安装基于 Python 的命令行工具 （接下来提到的部分程序使用 pip 来安装会很方便）。

来自 <https://github.com/jlevy/the-art-of-command-line/blob/master/README-zh.md>

日常使用

- 在 Bash 中，可以通过按 Tab 键实现自动补全参数，使用 ctrl-r 搜索命令行历史记录（按下按键之后，输入关键字便可以搜索，重复按下 ctrl-r 会向后查找匹配项，按下 Enter 键会执行当前匹配的命令，而按下右方向键会将匹配项放入当前行中，不会直接执行，以便做出修改）。
- 在 Bash 中，可以按下 ctrl-w 删除你键入的最后一个单词，ctrl-u 可以删除行内光标所在位置之前的内容，alt-b 和 alt-f 可以以单词为单位移动光标，ctrl-a 可以将光标移至行首，ctrl-e 可以将光标移至行尾，ctrl-k 可以删除光标至行尾的所有内容，ctrl-l 可以清屏。键入 man readline 可以查看 Bash 中的默认快捷键。内容有很多，例如 alt-. 循环地移向前一个参数，而 alt-* 可以展开通配符。
- 你喜欢的话，可以执行 set -o vi 来使用 vi 风格的快捷键，而执行 set -o emacs 可以把它改回来。
- 为了便于编辑长命令，在设置你的默认编辑器后（例如 export EDITOR=vim），ctrl-x ctrl-e 会打开一个编辑器来编辑当前输入的命令。在 vi 风格下快捷键则是 escape-v。
- 键入 history 查看命令行历史记录，再用 !n（n 是命令编号）就可以再次执行。其中有许多缩写，最有用的大概就是 !$， 它用于指代上次键入的参数，而 !! 可以指代上次键入的命令了（参考 man 页面中的“HISTORY

EXPANSION”）。不过这些功能，你也可以通过快捷键 ctrl-r 和 alt-. 来实现。

- cd 命令可以切换工作路径，输入 cd ~ 可以进入 home 目录。要访问你的 home 目录中的文件，可以使用前缀 ~（例如 ~/.bashrc）。在 sh 脚本里则用环境变量 $HOME 指代 home 目录的路径。
- 回到前一个工作路径：cd -。
- 如果你输入命令的时候中途改了主意，按下 alt-# 在行首添加 # 把它当做注释再按下回车执行（或者依次按下 ctrl-a， #， enter）。这样做的话，之后借助命令行历史记录，你可以很方便恢复你刚才输入到一半的命令。
- 使用 xargs （ 或 parallel）。他们非常给力。注意到你可以控制每行参数个数（-L）和最大并行数（-P）。如果你不确定它们是否会按你想的那样工作，先使用 xargs echo 查看一下。此外，使用 -I{} 会很方便。例如：

```
find .-name '*.py'|xargs grep some_function
cat hosts |xargs -I{} ssh root@{} hostname
```

- pstree -p 以一种优雅的方式展示进程树。
- 使用 pgrep 和 pkill 根据名字查找进程或发送信号（-f 参数通常有用）。
- 了解你可以发往进程的信号的种类。比如，使用 kill -STOP [pid] 停止一个进程。使用 man 7 signal 查看详细列表。
- 使用 nohup 或 disown 使一个后台进程持续运行。
- 使用 netstat -lntp 或 ss -plat 检查哪些进程在监听端口（默认是检查 TCP 端口；添加参数 -u 则检查 UDP 端口）或者 lsof -iTCP -sTCP:LISTEN -P -n (这也可以在 OS X 上运行)。
- lsof 来查看开启的套接字和文件。
- 使用 uptime 或 w 来查看系统已经运行多长时间。
- 使用 alias 来创建常用命令的快捷形式。例如：alias ll='ls -latr' 创建了一个新的命令别名 ll。
- 可以把别名、shell 选项和常用函数保存在 ~/.bashrc，具体看下这篇文章。这样做的话你就可以在所有 shell 会话中使用你的设定。
- 把环境变量的设定以及登陆时要执行的命令保存在 ~/.bash_profile。而对于从图形界面启动的 shell 和 cron 启动的 shell，则需要单独配置文件。
- 要想在几台电脑中同步你的配置文件（例如 .bashrc 和 .bash_profile），可以借助 Git。
- 当变量和文件名中包含空格的时候要格外小心。Bash 变量要用引号括起来，比如 "$FOO"。尽量使用 -0 或 -print0 选项以便用 NULL 来分隔文件名，例如 locate -0 pattern | xargs -0 ls -al 或 find / -print0 -type d | xargs -0 ls -al。如果 for 循环中循环访问的文件名含有空字符（空格、tab 等字符），只需

用 IFS=$'\n' 把内部字段分隔符设为换行符。

- 在 Bash 脚本中，使用 set -x 去调试输出（或者使用它的变体 set -v，它会记录原始输入，包括多余的参数和注释）。尽可能地使用严格模式：使用 set -e 令脚本在发生错误时退出而不是继续运行；使用 set -u 来检查是否使用了未赋值的变量；试试 set -o pipefail，它可以监测管道中的错误。当牵扯到很多脚本时，使用 trap 来检测 ERR 和 EXIT。一个好的习惯是在脚本文件开头这样写，这会使它能够检测一些错误，并在错误发生时中断程序并输出信息：

```
set-euo pipefail
trap"echo 'error: Script failed: see failed command above'"ERR
```

- 在 Bash 脚本中，子 shell（使用括号 (...)）是一种组织参数的便捷方式。一个常见的例子是临时地移动工作路径，代码如下：

```
#do something in current dir(cd /some/other/dir &&other-command)
#continue in original dir
```

- 在 Bash 中，变量有许多的扩展方式。${name:?error message} 用于检查变量是否存在。此外，当 Bash 脚本只需要一个参数时，可以使用这样的代码 input_file=${1:?usage: $0 input_file}。在变量为空时使用默认值：${name:-default}。如果你要在之前的例子中再加一个（可选的）参数，可以使用类似这样的代码 output_file=${2:-logfile}，如果省略了 $2，它的值就为空，于是 output_file 就会被设为 logfile。数学表达式：i=$(((i + 1) % 5))。序列：{1..10}。截断字符串：${var%suffix} 和 ${var#prefix}。例如，假设 var=foo.pdf，那么 echo ${var%.pdf}.txt 将输出 foo.txt。

- 使用括号扩展（{...}）来减少输入相似文本，并自动化文本组合。这在某些情况下会很有用，例如 mv foo.{txt,pdf} some-dir（同时移动两个文件），cp somefile{,.bak}（会被扩展成 cp somefile somefile.bak）或者 mkdir -p test-{a,b,c}/subtest-{1,2,3}（会被扩展成所有可能的组合，并创建一个目录树）。

- 通过使用 <(some command) 可以将输出视为文件。例如，对比本地文件 /etc/hosts 和一个远程文件：

```
diff /etc/hosts <(ssh somehost cat /etc/hosts)
```

- 编写脚本时，你可能会想要把代码都放在大括号里。缺少右括号的话，代码就会因为语法错误而无法执行。如果你的脚本是要放在网上分享供他人使用的，这样的写法就体现出它的好处了，因为这样可以防止下载不完全代码被执行。

```
{
#在这里写代码}
```

- 了解 Bash 中的“here documents”，例如 cat <<EOF ...。

- 在 Bash 中，同时重定向标准输出和标准错误：some-command >logfile 2>&1 或者 some-command &>logfile。通常，为了保证命令不会在标准输入里残留一个未关闭的文件句柄捆绑在你当前所在的终端上，在命令后添加 </dev/null 是一个好习惯。

- 使用 man ascii 查看具有十六进制和十进制值的ASCII表。man unicode，man utf-8，以及 man latin1 有助于你去了解通用的编码信息。

- 使用 screen 或 tmux 来使用多份屏幕，当你在使用 ssh 时（保存 session 信息）将尤为有用。而 byobu 可以为它们提供更多的信息和易用的管理工具。另一

个轻量级的 session 持久化解决方案是 dtach。

- ssh 中，了解如何使用 -L 或 -D（偶尔需要用 -R）开启隧道是非常有用的，比如当你需要从一台远程服务器上访问 web 页面。
- 对 ssh 设置做一些小优化可能是很有用的，例如这个 ~/.ssh/config 文件包含了防止特定网络环境下连接断开、压缩数据、多通道等选项：

```
TCPKeepAlive=yes
ServerAliveInterval=15
ServerAliveCountMax=6
Compression=yes
ControlMaster auto
ControlPath /tmp/%r@%h:%p
ControlPersist yes
```

- 一些其他的关于 ssh 的选项是与安全相关的，应当小心翼翼的使用。例如你应当只能在可信任的网络中启用 StrictHostKeyChecking=no，ForwardAgent=yes。
- 考虑使用 mosh 作为 ssh 的替代品，它使用 UDP 协议。它可以避免连接被中断并且对带宽需求更小，但它需要在服务端做相应的配置。
- 获取八进制形式的文件访问权限（修改系统设置时通常需要，但 ls 的功能不那么好用并且通常会搞砸），可以使用类似如下的代码：

```
stat -c '%A %a %n'/etc/timezone
```

- 使用 percol 或者 fzf 可以交互式地从另一个命令输出中选取值。
- 使用 fpp（PathPicker）可以与基于另一个命令(例如 git）输出的文件交互。
- 将 web 服务器上当前目录下所有的文件（以及子目录）暴露给你所处网络的所有用户，使用： python -m SimpleHTTPServer 7777 （使用端口 7777 和 Python 2）或python -m http.server 7777 （使用端口 7777 和 Python 3）。
- 以其他用户的身份执行命令，使用 sudo。默认以 root 用户的身份执行；使用 -u 来指定其他用户。使用 -i 来以该用户登录（需要输入_你自己的_密码）。
- 将 shell 切换为其他用户，使用 su username 或者 su - username。加入 - 会使得切换后的环境与使用该用户登录后的环境相同。省略用户名则默认为 root。切换到哪个用户，就需要输入_哪个用户的_密码。
- 了解命令行的 128K 限制。使用通配符匹配大量文件名时，常会遇到“Argument list too long”的错误信息。（这种情况下换用 find 或 xargs 通常可以解决。）
- 当你需要一个基本的计算器时，可以使用 python 解释器（当然你要用 python 的时候也是这样）。例如：

```
>>> 2+3
5
```

文件及数据处理

- 在当前目录下通过文件名查找一个文件，使用类似于这样的命令：find . -iname '*something*'。在所有路径下通过文件名查找文件，使用 locate something （但注意到 updatedb 可能没有对最近新建的文件建立索引，所以你可能无法定位到这些未被索引的文件）。
- 使用 ag 在源代码或数据文件里检索（grep -r 同样可以做到，但相比之下 ag 更加先进）。
- 将 HTML 转为文本：lynx -dump -stdin。
- Markdown，HTML，以及所有文档格式之间的转换，试试 pandoc。
- 当你要处理棘手的 XML 时候，xmlstarlet 算是上古时代流传下来的神器。
- 使用 jq 处理 JSON。
- 使用 shyaml 处理 YAML。
- 要处理 Excel 或 CSV 文件的话，csvkit 提供了 in2csv，csvcut，csvjoin，csvgrep 等方便易用的工具。
- 当你要处理 Amazon S3 相关的工作的时候，s3cmd 是一个很方便的工具而 s4cmd 的效率更高。Amazon 官方提供的 aws 以及 saws 是其他 AWS 相关工作的基础，值得学习。
- 了解如何使用 sort 和 uniq，包括 uniq 的 -u 参数和 -d 参数，具体内容在后文单行脚本节中。另外可以了解一下 comm。
- 了解如何使用 cut，paste 和 join 来更改文件。很多人都会使用 cut，但遗忘了 join。
- 了解如何运用 wc 去计算新行数（-l），字符数（-m），单词数（-w）以及字节数（-c）。
- 了解如何使用 tee 将标准输入复制到文件甚至标准输出，例如 ls -al | tee file.txt。
- 要进行一些复杂的计算，比如分组、逆序和一些其他的统计分析，可以考虑使用 datamash。
- 注意到语言设置（中文或英文等）对许多命令行工具有一些微妙的影响，比如排序的顺序和性能。大多数 Linux 的安装过程会将 LANG 或其他有关的变量设置为符合本地的设置。要意识到当你改变语言设置时，排序的结果可能会改变。明白国际化可能会使 sort 或其他命令运行效率下降*许多倍*。某些情况下（例如集合运算）你可以放心的使用 export LC_ALL=C 来忽略掉国际化并按照字节来判断顺序。
- 你可以单独指定某一条命令的环境，只需在调用时把环境变量设定放在命令的前面，例如 TZ=Pacific/Fiji date 可以获取斐济的时间。
- 了解如何使用 awk 和 sed 来进行简单的数据处理。 参阅 One-liners 获取示例。

- 替换一个或多个文件中出现的字符串：

```
perl -pi.bak -e 's/old-string/new-string/g' my-files-*.txt
```

- 使用 repren 来批量重命名文件，或是在多个文件中搜索替换内容。（有些时候 rename 命令也可以批量重命名，但要注意，它在不同 Linux 发行版中的功能并不完全一样。）

```
#将文件、目录和内容全部重命名 foo -> bar:repren --full --preserve-case --from foo --to bar .#还原所有备份文件 whatever.bak -> whatever:repren --renames --from '(.*)\.bak'--to '\1'*.bak
#用 rename 实现上述功能（若可用）:rename 's/\.bak$//'*.bak
```

- 根据 man 页面的描述，rsync 是一个快速且非常灵活的文件复制工具。它闻名于设备之间的文件同步，但其实它在本地情况下也同样有用。在安全设置允许下，用 rsync 代替 scp 可以实现文件续传，而不用重新从头开始。它同时也是删除大量文件的最快方法之一：

```
mkdir empty &&rsync -r --delete empty/ some-dir &&rmdir some-dir
```

- 若要在复制文件时获取当前进度，可使用 pv，pycp，progress，rsync --progress。若所执行的复制为block块拷贝，可以使用 dd status=progress。

- 使用 shuf 可以以行为单位来打乱文件的内容或从一个文件中随机选取多行。

- 了解 sort 的参数。显示数字时，使用 -n 或者 -h 来显示更易读的数（例如 du -h 的输出）。明白排序时关键字的工作原理（-t 和 -k）。例如，注意到你需要 -k1，1 来仅按第一个域来排序，而 -k1 意味着按整行排序。稳定排序（sort -s）在某些情况下很有用。例如，以第二个域为主关键字，第一个域为次关键字进行排序，你可以使用 sort -k1，1 | sort -s -k2，2。

- 如果你想在 Bash 命令行中写 tab 制表符，按下 ctrl-v [Tab] 或键入 $'\t' （后者可能更好，因为你可以复制粘贴它）。

- 标准的源代码对比及合并工具是 diff 和 patch。使用 diffstat 查看变更总览数据。注意到 diff -r 对整个文件夹有效。使用 diff -r tree1 tree2 | diffstat 查看变更的统计数据。vimdiff 用于比对并编辑文件。

- 对于二进制文件，使用 hd，hexdump 或者 xxd 使其以十六进制显示，使用 bvi，hexedit 或者 biew 来进行二进制编辑。

- 同样对于二进制文件，strings（包括 grep 等工具）可以帮助在二进制文件中查找特定比特。

- 制作二进制差分文件（Delta 压缩），使用 xdelta3。

- 使用 iconv 更改文本编码。需要更高级的功能，可以使用 uconv，它支持一些高级的 Unicode 功能。例如，这条命令移除了所有重音符号：

```
uconv -f utf-8 -t utf-8 -x '::Any-Lower; ::Any-NFD; [:Nonspacing Mark:] >; ::Any-NFC; '<input.txt >output.txt
```

- 拆分文件可以使用 split（按大小拆分）和 csplit（按模式拆分）。

- 操作日期和时间表达式，可以用 dateutils 中的 dateadd、datediff、strptime 等工具。

- 使用 zless、zmore、zcat 和 zgrep 对压缩过的文件进行操作。
- 文件属性可以通过 chattr 进行设置，它比文件权限更加底层。例如，为了保护文件不被意外删除，可以使用不可修改标记：sudo chattr +i /critical/directory/or/file
- 使用 getfacl 和 setfacl 以保存和恢复文件权限。例如：

```
getfacl -R /some/path >permissions.txt
setfacl --restore=permissions.txt
```

- 为了高效地创建空文件，请使用 truncate（创建稀疏文件），fallocate（用于 ext4，xfs，btrf 和 ocfs2 文件系统），xfs_mkfile（适用于几乎所有的文件系统，包含在 xfsprogs 包中），mkfile（用于类 Unix 操作系统，比如 Solaris 和 Mac OS）。

系统调试

- curl 和 curl -I 可以被轻松地应用于 web 调试中，它们的好兄弟 wget 也是如此，或者也可以试试更潮的 httpie。
- 获取 CPU 和硬盘的使用状态，通常使用使用 top（htop 更佳），iostat 和 iotop。而 iostat -mxz 15 可以让你获悉 CPU 和每个硬盘分区的基本信息和性能表现。
- 使用 netstat 和 ss 查看网络连接的细节。
- dstat 在你想要对系统的现状有一个粗略的认识时是非常有用的。然而若要对系统有一个深度的总体认识，使用 glances，它会在一个终端窗口中向你提供一些系统级的数据。
- 若要了解内存状态，运行并理解 free 和 vmstat 的输出。值得留意的是“cached”的值，它指的是 Linux 内核用来作为文件缓存的内存大小，而与空闲内存无关。
- Java 系统调试则是一件截然不同的事，一个可以用于 Oracle 的 JVM 或其他 JVM 上的调试的技巧是你可以运行 kill -3 <pid> 同时一个完整的栈轨迹和堆概述（包括 GC 的细节）会被保存到标准错误或是日志文件。JDK 中的 jps，jstat，jstack，jmap 很有用。SJK tools 更高级。
- 使用 mtr 去跟踪路由，用于确定网络问题。
- 用 ncdu 来查看磁盘使用情况，它比寻常的命令，如 du -sh *，更节省时间。
- 查找正在使用带宽的套接字连接或进程，使用 iftop 或 nethogs。
- ab 工具（Apache 中自带）可以简单粗暴地检查 web 服务器的性能。对于更复杂的负载测试，使用 siege。
- wireshark，tshark 和 ngrep 可用于复杂的网络调试。
- 了解 strace 和 ltrace。这俩工具在你的程序运行失败、挂起甚至崩溃，而你却不知道为什么或你想对性能有个总体的认识的时候是非常有用的。注意 profile 参数（-c）和附加到一个运行的进程参数 （-p）。

- 了解使用 ldd 来检查共享库。但是永远不要在不信任的文件上运行。
- 了解如何运用 gdb 连接到一个运行着的进程并获取它的堆栈轨迹。
- 学会使用 /proc。它在调试正在出现的问题的时候有时会效果惊人。比如：/proc/cpuinfo，/proc/meminfo，/proc/cmdline，/proc/xxx/cwd，/proc/xxx/exe，/proc/xxx/fd/，/proc/xxx/smaps（这里的 xxx 表示进程的 id 或 pid）。
- 当调试一些之前出现的问题的时候，sar 非常有用。它展示了 cpu、内存以及网络等的历史数据。
- 关于更深层次的系统分析以及性能分析，看看 stap（SystemTap），perf，以及 sysdig。
- 查看你当前使用的系统，使用 uname，uname -a（Unix / kernel 信息）或者 lsb_release -a（Linux 发行版信息）。
- 无论什么东西工作得很欢乐（可能是硬件或驱动问题）时可以试试 dmesg。
- 如果你删除了一个文件，但通过 du 发现没有释放预期的磁盘空间，请检查文件是否被进程占用： lsof | grep deleted | grep "filename-of-my-big-file"

单行脚本

一些命令组合的例子：

- 当你需要对文本文件做集合交、并、差运算时，sort 和 uniq 会是你的好帮手。具体例子请参照代码后面的，此处假设 a 与 b 是两内容不同的文件。这种方式效率很高，并且在小文件和上 G 的文件上都能运用（注意尽管在 /tmp 在一个小的根分区上时你可能需要 -T 参数，但是实际上 sort 并不被内存大小约束），参阅前文中关于 LC_ALL 和 sort 的 -u 参数的部分。
  sort a b |uniq >c #c 是 a 并 bsort a b |uniq -d >c #c 是 a 交 bsort a b b |uniq -u >c #c 是 a - b
- 使用 grep . *（每行都会附上文件名）或者 head -100 *（每个文件有一个标题）来阅读检查目录下所有文件的内容。这在检查一个充满配置文件的目录（如 /sys、/proc、/etc）时特别好用。
- 计算文本文件第三列中所有数的和（可能比同等作用的 Python 代码快三倍且代码量少三倍）：

  awk '{ x += $3 } END { print x }'myfile
- 如果你想在文件树上查看大小/日期，这可能看起来像递归版的 ls -l 但比 ls -lR 更易于理解：
  find .-type f -ls
- 假设你有一个类似于 web 服务器日志文件的文本文件，并且一个确定的值只会出现在某些行上，假设一个 acct_id 参数在 URI 中。如果你想计算出每个 acct_id 值有多少次请求，使用如下代码：
  egrep -o 'acct_id=[0-9]+'access.log |cut -d= -f2 |sort |uniq -c |sort -rn
- 要持续监测文件改动，可以使用 watch，例如检查某个文件夹中文件的改变，可以用 watch -d -n 2 'ls -rtlh | tail'；或者在排查 WiFi 设置故障时要监测网络

设置的更改，可以用 watch -d -n 2 ifconfig。

- 运行这个函数从这篇文档中随机获取一条技巧（解析 Markdown 文件并抽取项目）：

```
functiontaocl(){
  curl -s https://raw.githubusercontent.com/jlevy/the-art-of-command-line/master/README-zh.md|pandoc -f markdown -t html |iconv -f 'utf-8'-t 'unicode'|xmlstarlet fo --html --dropdtd |xmlstarlet sel -t -v "(html/body/ul/li[count(p)>0])[$RANDOMmod last()+1]"|xmlstarlet unesc |fmt -80
}
```

冷门但有用

- expr：计算表达式或正则匹配
- m4：简单的宏处理器
- yes：多次打印字符串
- cal：漂亮的日历
- env：执行一个命令（脚本文件中很有用）
- printenv：打印环境变量（调试时或在写脚本文件时很有用）
- look：查找以特定字符串开头的单词或行
- cut，paste 和 join：数据修改
- fmt：格式化文本段落
- pr：将文本格式化成页 / 列形式
- fold：包裹文本中的几行
- column：将文本格式化成多个对齐、定宽的列或表格
- expand 和 unexpand：制表符与空格之间转换
- nl：添加行号
- seq：打印数字
- bc：计算器
- factor：分解因数
- gpg：加密并签名文件
- toe：terminfo 入口列表
- nc：网络调试及数据传输
- socat：套接字代理，与 netcat 类似

- slurm：网络流量可视化
- dd：文件或设备间传输数据
- file：确定文件类型
- tree：以树的形式显示路径和文件，类似于递归的 ls
- stat：文件信息
- time：执行命令，并计算执行时间
- timeout：在指定时长范围内执行命令，并在规定时间结束后停止进程
- lockfile：使文件只能通过 rm -f 移除
- logrotate： 切换、压缩以及发送日志文件
- watch：重复运行同一个命令，展示结果并 / 或高亮有更改的部分
- when-changed：当检测到文件更改时执行指定命令。参阅 inotifywait 和 entr。
- tac：反向输出文件
- shuf：文件中随机选取几行
- comm：一行一行的比较排序过的文件
- strings：从二进制文件中抽取文本
- tr：转换字母
- iconv 或 uconv：文本编码转换
- split 和 csplit：分割文件
- sponge：在写入前读取所有输入，在读取文件后再向同一文件写入时比较有用，例如 grep -v something some-file | sponge some-file
- units：将一种计量单位转换为另一种等效的计量单位（参阅 /usr/share/units/definitions.units）
- apg：随机生成密码
- xz：高比例的文件压缩
- ldd：动态库信息
- nm：提取 obj 文件中的符号
- ab 或 wrk：web 服务器性能分析
- strace：调试系统调用
- mtr：更好的网络调试跟踪工具

- cssh：可视化的并发 shell
- rsync：通过 ssh 或本地文件系统同步文件和文件夹
- wireshark 和 tshark：抓包和网络调试工具
- ngrep：网络层的 grep
- host 和 dig：DNS 查找
- lsof：列出当前系统打开文件的工具以及查看端口信息
- dstat：系统状态查看
- glances：高层次的多子系统总览
- iostat：硬盘使用状态
- mpstat： CPU 使用状态
- vmstat： 内存使用状态
- htop：top 的加强版
- last：登入记录
- w：查看处于登录状态的用户
- id：用户/组 ID 信息
- sar：系统历史数据
- iftop 或 nethogs：套接字及进程的网络利用情况
- ss：套接字数据
- dmesg：引导及系统错误信息
- sysctl： 在内核运行时动态地查看和修改内核的运行参数
- hdparm：SATA/ATA 磁盘更改及性能分析
- lsblk：列出块设备信息：以树形展示你的磁盘以及磁盘分区信息
- lshw，lscpu，lspci，lsusb 和 dmidecode：查看硬件信息，包括 CPU、BIOS、RAID、显卡、USB设备等
- lsmod 和 modinfo：列出内核模块，并显示其细节
- fortune，ddate 和 sl：额，这主要取决于你是否认为蒸汽火车和莫名其妙的名人名言是否“有用”

仅限 OS X 系统

以下是*仅限于* OS X 系统的技巧。

- 用 brew （Homebrew）或者 port （MacPorts）进行包管理。这些可以用来在 OS X 系统上安装以上的大多数命令。
- 用 pbcopy 复制任何命令的输出到桌面应用，用 pbpaste 粘贴输入。
- 若要在 OS X 终端中将 Option 键视为 alt 键（例如在上面介绍的 alt-b、alt-f 等命令中用到），打开 偏好设置 -> 描述文件 -> 键盘 并勾选“使用 Option 键作为 Meta 键”。
- 用 open 或者 open -a /Applications/Whatever.app 使用桌面应用打开文件。
- Spotlight：用 mdfind 搜索文件，用 mdls 列出元数据（例如照片的 EXIF 信息）。
- 注意 OS X 系统是基于 BSD UNIX 的，许多命令（例如 ps，ls，tail，awk，sed）都和 Linux 中有微妙的不同（ Linux 很大程度上受到了 System V-style Unix 和 GNU 工具影响）。你可以通过标题为 "BSD General Commands Manual" 的 man 页面发现这些不同。在有些情况下 GNU 版本的命令也可能被安装（例如 gawk 和 gsed 对应 GNU 中的 awk 和 sed ）。如果要写跨平台的 Bash 脚本，避免使用这些命令（例如，考虑 Python 或者 perl ）或者经过仔细的测试。
- 用 sw_vers 获取 OS X 的版本信息。

仅限 Windows 系统

以下是*仅限于* Windows 系统的技巧。

在 Winodws 下获取 Unix 工具

- 可以安装 Cygwin 允许你在 Microsoft Windows 中体验 Unix shell 的威力。这样的话，本文中介绍的大多数内容都将适用。
- 在 Windows 10 上，你可以使用 Bash on Ubuntu on Windows，它提供了一个熟悉的 Bash 环境，包含了不少 Unix 命令行工具。好处是它允许 Linux 上编写的程序在 Windows 上运行，而另一方面，Windows 上编写的程序却无法在 Bash 命令行中运行。
- 如果你在 Windows 上主要想用 GNU 开发者工具（例如 GCC），可以考虑 MinGW 以及它的 MSYS 包，这个包提供了例如 bash，gawk，make 和 grep 的工具。MSYS 并不包含所有可以与 Cygwin 媲美的特性。当制作 Unix 工具的原生 Windows 端口时 MinGW 将特别地有用。
- 另一个在 Windows 下实现接近 Unix 环境外观效果的选项是 Cash。注意在此环境下只有很少的 Unix 命令和命令行可用。

实用 Windows 命令行工具

- 可以使用 wmic 在命令行环境下给大部分 Windows 系统管理任务编写脚本以及执行这些任务。

- Windows 实用的原生命令行网络工具包括 ping，ipconfig，tracert，和 netstat。
- 可以使用 Rundll32 命令来实现许多有用的 Windows 任务 。

Cygwin 技巧

- 通过 Cygwin 的包管理器来安装额外的 Unix 程序。
- 使用 mintty 作为你的命令行窗口。
- 要访问 Windows 剪贴板，可以通过 /dev/clipboard。
- 运行 cygstart 以通过默认程序打开一个文件。
- 要访问 Windows 注册表，可以使用 regtool。
- 注意 Windows 驱动器路径 C:\ 在 Cygwin 中用 /cygdrive/c 代表，而 Cygwin 的 / 代表 Windows 中的 C:\cygwin。要转换 Cygwin 和 Windows 风格的路径可以用 cygpath。这在需要调用 Windows 程序的脚本里很有用。
- 学会使用 wmic，你就可以从命令行执行大多数 Windows 系统管理任务，并编成脚本。
- 要在 Windows 下获得 Unix 的界面和体验，另一个办法是使用 Cash。需要注意的是，这个环境支持的 Unix 命令和命令行参数非常少。
- 要在 Windows 上获取 GNU 开发者工具（比如 GCC）的另一个办法是使用 MinGW 以及它的 MSYS 软件包，该软件包提供了 bash、gawk、make、grep 等工具。然而 MSYS 提供的功能没有 Cygwin 完善。MinGW 在创建 Unix 工具的 Windows 原生移植方面非常有用。

更多资源

- awesome-shell：一份精心组织的命令行工具及资源的列表。
- awesome-osx-command-line：一份针对 OS X 命令行的更深入的指南。
- Strict mode：为了编写更好的脚本文件。
- shellcheck：一个静态 shell 脚本分析工具，本质上是 bash / sh / zsh 的 lint。
- Filenames and Pathnames in Shell：有关如何在 shell 脚本里正确处理文件名的细枝末节。
- Data Science at the Command Line：用于数据科学的一些命令和工具，摘自同名书籍。

来自 <https://github.com/jlevy/the-art-of-command-line/blob/master/README-zh.md>

查看磁盘I/O速度

2023年1月18日 10:09

iostat -dmx 1 (每秒刷新一次)

img镜像制作

2023年1月18日 10:10

find . | cpio -o -H newc | gzip -9 > img

img文件查看

2023年1月18日 10:10

1. 有文件系统的img直接挂载
2. 加后缀gz,执行

gunzip file.img.gz

cpio -i -d -H newc --no-absolute-filenames --quiet < file.img

查看网卡接入状态

2023年1月18日 10:11

ethtool 网口名称； 可以查看网口连接状态，如是否连接 link detected

查看一个模块里的符号情况

2023年1月18日 10:11

可以通过nm -l xx.ko来

网络配置

2023年1月18日 10:11

centos	/etc/sysconfig/network-scripts
Ubuntu	/etc/plan/

堆栈代码行定位命令

2023年1月18日 10:12

```
eu-addr2line -f -e ./rtl.ko rtl_init+0xed
```

手动释放缓存

2023年1月18日 10:14

Linux释放内存的命令：
sync
echo 1 > /proc/sys/vm/drop_caches

drop_caches的值可以是0-3之间的数字，代表不同的含义：

0：不释放（系统默认值）

1：释放页缓存

2：释放dentries和inodes

3：释放所有缓存

释放完内存后改回去让系统重新自动分配内存。
echo 0 >/proc/sys/vm/drop_caches

来自 <https://blog.csdn.net/hellojoy/article/details/80760010>

创建文件的方法

2023年1月18日 10:15

1.touch

1）创建一个文件

touch test.txt

2）创建两个文件

touch test1.txt test2.txt

3)创建多个文件夹（如创建2000个）

touch test{0001.2000}.txt

4）更改文件创建的时间为当前时间（文件已存在）

touch test.txt

2.vi和vim

vi test.txt
vim test.txt

3.使用>或>>

> 直接覆盖原文件

>> 在文末增加内容

结合is，grep，echo创建

4.cp

只要文件是新的就算创建文件

5.cat

结合>、>>或eof或exit创建

查看分区的文件系统类型

2023年1月18日 10:16

df -T

搜索目录包括子目录的文件中的内容

2023年2月10日 15:15

```
grep -nr "/etc/init.d/swpoweroff start" *
```

Tcpdump

2023年2月15日 11:11

```
[root@localhost raid]# tcpdump --help
tcpdump version 4.9.0
libpcap version 1.5.3
OpenSSL 1.0.1e-fips 11 Feb 2013
Usage: tcpdump [-aAbdDefhHIJKlLnNOpqStuUvxX#] [ -B size ] [ -c count ]
        [ -C file_size ] [ -E algo:secret ] [ -F file ] [ -G seconds ]
        [ -i interface ] [ -j tstamptype ] [ -M secret ] [ --number ]
        [ -Q|-P in|out|inout ]
        [ -r file ] [ -s snaplen ] [ --time-stamp-precision precision ]
        [ --immediate-mode ] [ -T type ] [ --version ] [ -V file ]
        [ -w file ] [ -W filecount ] [ -y datalinktype ] [ -z postrotate-command ]
        [ -Z user ] [ expression ]
```

第一个选项是不带参数的参数，最后一个选项是表达式，且是正则表达式；其余选项是带参数的选项，具体意义参考下面。

常用	-x以十六进制显示报文内容 -A以ASCII码显示保温内容

一：命令介绍：

tcpdump，用简单的语言概括就是dump the traffic on a network，是一个运行在linux平台可以根据使用者需求对网络上传输的数据包进行捕获的抓包工具，windows平台有sniffer等工具，tcpdump可以将网络中传输的数据包的“包头”全部捕获过来进程分析，其支持网络层、特定的传输协议、数据发送和接收的主机、网卡和端口的过滤，并提供and、or、not等语句进行逻辑组合捕获数据包或去掉不用的信息。

1：命令选项和捕获主机对到主机的数据包：

1.1：命令选项：

-a #将网络地址和广播地址转变成名字

-A #以ASCII格式打印出所有分组，并将链路层的头最小化

-b #数据链路层上选择协议，包括ip/arp/rarp/ipx都在这一层

-c #指定收取数据包的次数，即在收到指定数量的数据包后退出tcpdump

-d #将匹配信息包的代码以人们能够理解的汇编格式输出

-dd #将匹配信息包的代码以c语言程序段的格式输出

-ddd #将匹配信息包的代码以十进制的形式输出

-D #打印系统中所有可以监控的网络接口

-e #在输出行打印出数据链路层的头部信息

-f #将外部的Internet地址以数字的形式打印出来，即不显示主机名

-F #从指定的文件中读取表达式，忽略其他的表达式

-i #指定监听网络接口

-l #使标准输出变为缓冲形式，可以数据导出到文件

-L #列出网络接口已知的数据链路

-n #不把网络地址转换为名字

-N 不输出主机名中的域名部分，例如www.baidu.com只输出www

-nn #不进行端口名称的转换

-P #不将网络接口设置为混杂模式

-q #快速输出，即只输出较少的协议信息

-r #从指定的文件中读取数据，一般是-w保存的文件

-w #将捕获到的信息保存到文件中，且不分析和打印在屏幕

-s #从每个组中读取在开始的snaplen个字节，而不是默认的68个字节

-S #将tcp的序列号以绝对值形式输出，而不是相对值

-T #将监听到的包直接解析为指定的类型的报文，常见的类型有rpc（远程过程调用）和snmp（简单网络管理协议）

-t #在输出的每一行不打印时间戳

-tt #在每一行中输出非格式化的时间戳

-ttt #输出本行和前面以后之间的时间差

-tttt #在每一行中输出data处理的默认格式的时间戳

-u #输出未解码的NFS句柄

-v #输出稍微详细的信息，例如在ip包中可以包括ttl和服务类型的信息

-vv#输出相信的保报文信息

1.1.1：#tcpdump的表达式：

表达式是一个正则表达式，tcpdump利用它作为过滤报文的条件，如果一个报文满足表达式的条件，则这个报文将会被捕获。如果没有给出任何条件，则网络上所有的信息包 将会被截获，在表达式中一般如下几种类型的关键字：

a)：关于数据类型的关键字：

包括host、port、net，例如host 192.168.1.1表示这是一台主机，net 192.168.0.0表示这是一个网络地址，port 22指明端口号是22，如果没有指明类型，则默认的类型是host。

b)：数据传输方向的关键字：

包括src、dst、dst or src、dst and src，这些关键字指明了传输的方向，比如src 192.168.1.1说明数据包源地址是192.168.1.1，dst net 192.168.0.0指明目的网络地址是192.168.0.0，默认是监控主机对主机的src和dst，即默认监听本机和目标主机的所有数据。

c)：协议关键字：

包括ip、arp、rarp、tcp、udp等，

d)：其他关键字：

运算类型的：or、and、not、!

辅助功能型的：gateway、less、broadcast、greater

1.2：# tcpdump 默认捕获方式

#默认监听在第一块网卡上，监听所有经过此网卡通过的数据包

1.3：# tcpdump -i eth0 监听指定网卡eth0的所有传输数据包：

1.4：#tcpdump -i eth0 host 192.168.56.1 #捕获主机192.168.56.1经过本机网卡eth0的所有数据包（也可以是主机名，但要求可以解析出来IP地址）

1.5：#tcpdump host 192.168.56.209 and \(192.168.56.210 or 192.168.56.211 \) #

捕获主机 192.168.56.209 和主机192.168.56.210或192.168.56.211的所有通信数据包

1.6：#tcpdump ip host node9 and not www.baidu.com #捕获主机node9与其他主机之间（不包括www.baidu.com）通信的ip数据包

1.7：#tcpdump ip host node9 and ! www.baidu.com #捕获node9与其他所有主机的通信数据包（不包括www.baidu.com）

1.8：#tcpdump -i eth0 src node10 #捕获源主机node10发送的所有的经过eth0网卡的所有数据包

1.9：##tcpdump -i eth0 dst host www.baidu.com #捕获所有发送到主机www.baidu.com的数据包

1.9.1:监听主机192.168.56.1和192.168.56.210之间ip协议的80端口的且排除www.baidu.com通信的所有数据包：

tcpdump ip dst 192.168.56.1 and src 192.168.56.210 and port 80 and host ! www.baidu.com #也可以写成tcpdump ip dst 192.168.56.1 and src 192.168.56.210 and port 80 and host not www.baidu.com，即not和！都是相同的取反的意思

1.9.2：捕获arp的协议数据包：

#tcpdump arp #监控指定主机的通信数据包与1.9.1方式相同

2：捕获指定主机和端口的数据包：

2.1：#tcpdump tcp port 22 and host 192.168.56.210 #捕获主机192.168.56.210接收和发出的tcp协议的ssh的数据包：

2.2：# tcpdump udp port 53 #监听本机udp的53端口的数据包，udp是dns协议的端口，这也是一个dns域名解析的完整过程

总结：tcpdump的语法类似于mysql查询语句，可以指定各种查询的条件进行组合，还可以进行与或非的条件判断进行更精确的数据搜集，语法如下：

#tcpdump ［协议类型］［源或目标］［主机名称或IP］ ［or/and/not/!条件组合］［源或目标］［主机名或IP］ ［or/and/not/!条件组合］［端口］［端口号］…… ［or/and/not/!条件组合］［条件］

#tcpdump ip dst 192.168.56.1 and src 192.168.56.210 and port 80 and host ! www.baidu.com

来自 <https://www.cnblogs.com/howhy/p/6396664.html>

查看编译器支持的C++版本

2022年12月11日 11:41

gcc -E -dM - </dev/null | grep "STDC_VERSION"

#define __STDC_VERSION__ 201710L
root@weisonxs:/usr/lib/vcpkg/packages/jsoncpp_x64-linux/lib#

查看服务

2023年3月29日 16:53

在Linux中查看服务的命令取决于使用的发行版和服务管理工具。以下是一些可能有用的命令：

1. 通过systemd管理服务的发行版（如Ubuntu 16.04+、CentOS 7+等）：

- 查看所有服务： `systemctl list-unit-files --type=service`
- 查看正在运行的服务： `systemctl list-units --type=service`
- 查看特定服务的状态： `systemctl status <service_name>`
- 启动服务： `systemctl start <service_name>`
- 停止服务： `systemctl stop <service_name>`
- 重启服务： `systemctl restart <service_name>`
- 禁止服务开机自启： `systemctl disable <service_name>`
- 允许服务开机自启： `systemctl enable <service_name>`

2. 通过SysV init管理服务的发行版（如Red Hat Enterprise Linux 6及以下版本，以及一些其他发行版）：

- 查看所有服务： `chkconfig --list`
- 查看特定服务的状态： `service <service_name> status`
- 启动服务： `service <service_name> start`
- 停止服务： `service <service_name> stop`
- 重启服务： `service <service_name> restart`
- 禁止服务开机自启： `chkconfig <service_name> off`
- 允许服务开机自启： `chkconfig <service_name> on`

来自 <https://www.aiai.zone/#/chat/1680058804112>

SSH登录锁定解锁

2023年4月11日 17:53

【SSH】Suse Linux 10 ssh多次登录失败锁定用户及解锁

原创 Linux操作系统 作者：阳冬晓 时间：2010-11-01 15:19:41 13157 0

1、修改配置文件

vi /etc/pam.d/sshd

增加

auth required pam_tally.so deny=3 （deny=N为登录失败N次后锁定用户）

2、在客户端测试

ssh user@192.168.1.1（user为服务器端的用户，192.168.1.1为主机IP地址）

3、3次后再次测试并用正确密码登录会发现仍不能登录

4、解除锁定方法

在服务器端以root用户登录

执行命令：

faillog -a ////查看用户登录错误次数

如果超过三次的话，用户不能登录并且此后登录用户错误登录次数还是会增加。

在登录错误次数不满三次时，登录成功后，则这个用户登录错误值将清零，退出后重新ssh登录将采用新的计数。

faillog -u user -r ////清空指定用户user的错误登录次数

faillog -r //// 清空所有用户错误登录次数

来自 <http://blog.itpub.net/24376892/viewspace-677151/>

命令重定向操作符

2023年4月19日 14:49

Linux后台的print打印被重定向哪里输出

2023/4/19 14:48:25

Linux 中后台打印输出（print）通常是指程序或脚本在后台运行时产生的标准输出和标准错误输出。这些输出可以使用重定向操作符 "> " 和 "2> " 将其分别重定向到文件或其他设备中。

如果将标准输出重定向到文件，可以使用如下命令：

复制代码

```
$ command > output.txt &
```

其中，command 表示要在后台运行的命令，">" 表示输出重定向操作符，output.txt 表示要将标准输出重定向的文件名。"&" 表示将命令放入后台运行。

如果将标准错误输出重定向到文件，可以使用如下命令：

复制代码

```
$ command 2> error.txt &
```

其中，"2>" 表示标准错误输出重定向操作符，error.txt 表示要将标准错误输出重定向的文件名。

如果将标准输出和标准错误输出都重定向到文件，可以使用如下命令：

复制代码

```
$ command > output.txt 2> error.txt &
```

其中，">" 表示标准输出重定向操作符，"2>" 表示标准错误输出重定向操作符，output.txt 表示要将标准输出重定向的文件名，error.txt 表示要将标准错误输出重定向的文件名。

注	“2>” 整体是一个符号

顺序	使用重定向操作符可以将输出重定向到文件或其他位置，影响输出的顺序。下面是一些常见的重定向操作符： • >：将标准输出重定向到文件，会覆盖文件原有内容。 • >>：将标准输出重定向到文件，以追加模式添加到文件末尾。 • 2>：将标准错误重定向到文件。 • 2>>：将标准错误重定向到文件，以追加模式添加到文件末尾。 • &> 或 &>>：将标准输出和标准错误同时重定向到文件。 如果只使用 > 或 >> 进行重定向，只会处理标准输出，标准错误仍然会在终端上显示。要重定向标准错误，需要使用 2> 或 2>>。 因此，输出的顺序将受到命令在执行过程中输出到标准输出和标准错误的顺序以及使用的重定向操作符的影响。 来自 <https://chat.openai.com/>

系统的开机时间

2023年4月20日 20:05

在Linux中，可以使用命令uptime来查看系统的开机时间。该命令将显示当前时间、系统已运行的时间以及平均负载等信息。另外，也可以使用命令who -b来查看系统的启动时间。

来自 <https://mirrorchat.extkj.cn/#/chat/1681886702802>

搜索目录中的文件内容

2023年6月30日 11:36

如果你希望在所有目录中搜索文件内容，你可以使用`find`命令结合`grep`命令来完成。以下是一个示例命令：

```bash
find / -type f -exec grep -H "example" {} +
```

这个命令将从根目录开始递归地搜索所有文件，并在找到的文件中查找包含"example"的行。`-type f`选项指定只搜索常规文件，而不搜索目录或其他特殊文件。`-exec`选项允许在每个匹配的文件上执行后续命令。

在上述命令中，`grep -H "example" {} +`用于在每个匹配的文件中查找包含"example"的行，并显示匹配的内容和文件名。`-H`选项用于显示文件名。

请注意，这种方式可能会搜索整个文件系统，需要一定的时间和系统资源。如果你只想在特定目录中搜索，可以将根目录`/`替换为你想要搜索的目录路径。

触发oops重启系统

2023年8月1日 9:59

echo c > /proc/sysrq-trigger

查看CPU核心信息

2023年8月16日 15:18

在Linux系统中，你可以使用多种命令来查看CPU核心的情况。以下是一些常用的命令：

1. **lscpu**: 这个命令会显示关于CPU的详细信息，包括核心数量、线程数、架构等。

```bash
lscpu
```

2. **nproc**: 这个命令会显示系统上可用的CPU核心数量。

```bash
nproc
```

3. **cat /proc/cpuinfo**: 这个命令会显示CPU的详细信息，包括每个核心的信息。

```bash
cat /proc/cpuinfo
```

4. **top / htop**: 这些命令可以动态地显示系统的进程和资源使用情况，包括CPU使用情况。

```bash
top
# 或者使用 htop，如果已经安装的话
htop
```

5. **mpstat**: 这个命令可以显示多核CPU的使用情况，包括每个核心的使用率等。

```bash
mpstat -P ALL
```

6. **htop**: 这是一个交互式的进程查看工具，可以显示实时的CPU使用情况以及各个进程的资源消耗情况。

```bash
htop
```

请注意，命令的可用性可能会因不同的Linux发行版而有所不同。在使用这些命令时，你可以根据自己的需求选择合适的方式来查看CPU核心的情况。

modprobe

2023年10月27日 9:55

在Linux中，有一些内核模块是相互依赖的，因此当加载一个模块时，可能会自动加载其他相关的模块，以满足依赖关系。这是由模块间的依赖关系和内核的自动解析功能来实现的。您可以使用`modprobe`命令来加载模块，并它会自动处理依赖关系。

以下是一些与加载模块和处理依赖关系相关的常用命令：

1. **加载模块：** 使用`modprobe`命令来加载模块，例如：

```
modprobe <module_name>
```

2. **查看已加载的模块：** 使用`lsmod`命令来查看已加载的模块列表，例如：

```
lsmod
```

3. **查看模块的依赖关系：** 使用`modinfo`命令来查看模块的信息，包括其依赖关系，例如：

```
modinfo <module_name>
```

4. **手动加载依赖模块：** 如果您想手动加载一个模块及其依赖模块，可以使用`modprobe`命令并指定`-a`选项，例如：

```
modprobe -a <module_name>
```

5. **自动加载所有依赖：** 如果您希望加载一个模块以及其所有依赖模块，可以使用`-a`选项，例如：

```
modprobe -a -r <module_name>
```

请注意，具体的模块和依赖关系取决于您的Linux发行版和内核版本。您可以在`/lib/modules/<kernel_version>/`目录中找到内核模块，并在那里查看模块的依赖关系和相关信息。

关于exec函数簇命名意义

2022年8月30日 14:30

后缀是否有p	有p表示待执行的程序文件是由文件名指定否则为路径名
后缀是否含有l或v	有l（list）表示新程序参数是列出来的，有v表示新程序由指针数组引用
后缀是否有e	有e表示指定新环境，没有则使用调用进程环境

execlp	execl	execle
execvp	execv	execve

未命名

2023年5月16日 16:23

spin_lock_irqsave和spin_lock有什么区别

2023/5/16 16:24:37

spin_lock_irqsave()和spin_lock()都是用于实现自旋锁的函数，但它们之间的区别在于**前者会禁止中断**。具体来说，spin_lock_irqsave()在获取自旋锁时会将当前CPU的中断状态保存起来，并将中断禁止，以避免中断与自旋锁竞争资源造成死锁。而spin_lock()则不会禁止中断，在多处理器系统中可能会出现中断竞争问题。因此，在需要保护共享资源同时需要避免中断干扰时，应该使用spin_lock_irqsave()；否则可以使用spin_lock()。

来自 <https://mirrorchat.extkj.cn/>

语言基础

2021年4月1日 17:21

书籍：
《深入理解C++11：C++11新特性解析与应用》
《深入应用C++11：代码优化与工程级应用》
《C++17完全指南》
《Cpp 17 in Detail》

https://pan.baidu.com/s/1Af_6-bkugcTFljpIoeIk7Q
密码：mltg
（已保存到我的网盘）

项目

2021年4月8日 23:25

1.filezilla

一个开源的FTP软件，其源码下载地址:

https://svn.filezilla-project.org/svn/FileZilla3/trunk

需要svn工具来下载，安装好svn工具后，在svn界面中checkout上述地址或者使用如下命令下载：

svn co https://svn.filezilla-project.org/svn/FileZilla3/trunkfilezilla

2.uWebSocket网络库

下载地址： https://github.com/uNetworking/uWebSockets

3.TeamTalk的pc端

蘑菇街开源的一款用于企业内部的即时通信工具，其下载地址是：

https://github.com/balloonwj/TeamTalk/tree/master/win-client

4.Flamingo IM

https://github.com/balloonwj/flamingoballoonwj/flamingo

算法与数据结构

2021年4月8日 23:34

说到 算法 和数据结构，对于社招人士和对于应届生一般是不一样的，对于大的互联网公司和一般的小的企业也是不一样的。下面根据我当面试官面试别人和找工作被别人面试经验来谈一谈。

先说考察的内容，除了一些特殊的岗位，常见的 算法 和数据结构面试问题有如下：

排序 （常考的 排序 按频率考 排序 为：快速 排序 > 冒泡 排序 > 归并 排序 > 桶 排序 ）
一般对于对 算法 基础有要求的公司，如果你是应届生或者工作经验在一至三年内，以上 算法 如果写不出来，给面试官的影响会非常不好，甚至直接被 pass 掉。对于工作三年以上的社会人士，如果写不出来，但是能分析出其 算法 复杂度、最好和最坏的情况下的复杂度，说出 算法 大致原理，在多数面试官面前也可以过的。 注意，如果你是学生，写不出来或者写的不对，基本上面试过不了。

二分查找
二分查找 的 算法 尽量要求写出来。当然，大多数面试官并不会直接问你 二分查找 ，而是结合具体的场景，例如如何求一个数的平方根，这个时候你要能想到是 二分查找 。我在2017年年底，面试agora时，面试官问了一个问题：如何从所有很多的ip地址中快速找个某个ip地址。
链表
无论是应届生还是工作年限不长的社会人士，琏表常见的操作一定要熟练写出来，如 链表 的查找、定位、反转、连接等等。还有一些经典的问题也经常被问到，如两个 链表 如何判断有环（我在2017年面试 饿了么 二面、上海黄金交易所一面被问过）。 链表 的问题一般不难，但是 链表 的问题存在非常多的“ 坑”，如很多人不注意边界检查、空 链表 、返回一个 链表 的函数应该返回 链表 的头指针等等。
队列与栈
对于应届生来说一般这一类问的比较少，但是对于社会人士尤其是中高级岗位开发，会结合相关的问题问的比较多，例如让面试者利用队列写一个多线程下的生产者和消费者程序，全面考察的多线程的资源同步与竞态问题（下文介绍多线程面试题时详细地介绍）。
栈一般对于基础要求高的面试，会结合函数调用实现来问。即函数如何实现的，包括函数的调用的几种常见调用方式、参数的入栈顺序、内存栈在地址从高向低扩展、栈帧指针和栈顶指针的位置、函数内局部变量在栈中的内存分布、函数调用结束后，调用者和被调用者谁和如何清理栈等等。某年面试 京东 一基础部门，面试官让写从0加到100这样一个求和 算法 ，然后写其汇编代码。
哈希表
哈希表 是考察最多的数据结构之一。常见的问题有哈希冲突的检测、让面试者写一个哈希插

入函数等等。基本上一场面试下来不考察 红黑树 基本上就会问 哈希表 ， 而且问题可浅可深。我印象比较深刻的是，当年面试 百度 广告推荐部门时，二面问的一些关于 哈希表 的问题。当时面试官时先问的 链表 ， 接着问的哈希冲突的解决方案，后来让写一个哈希插入 算法 ， 这里需要注意的是，你的 算法 中插入的元素一定要是通用元素，所以对于 C++ 或者 Java 语言，一定要使用模板这一类参数作为哈希插入 算法 的对象。然后，就是 哈希表 中多个元素冲突时，某个位置的元素使用 链表 往后穿成一串的方案。最终考察 linux 下 malloc（下面的ptmalloc） 函数在频繁调用造成的内存碎片问题，以及开源方案解决方案 tcmalloc 和 jemalloc。总体下来，面试官是一步步引导你深入。（有兴趣的读者可以自行搜索，网上有很多相关资料）

树

面试高频的树是 红黑树 ， 也有一部分是B树（B+树）。

红黑树 一般的问的深浅不一，大多数面试官只要能说出 红黑树 的概念、左旋右旋的方式、分析出查找和插入的平均 算法 复杂度和最好最坏时的 算法 复杂度，并不要写面试者写出具体代码实现。一般 C++ 面试问 stl 的map，java 面试问 TreeMap 基本上就等于开始问你 红黑树 了，要有心里准备。笔者曾经面试 爱奇艺 被问过 红黑树 。

B树一般不会直接问，问的最多的形式是通过问 MySQL 索引实现原理（数据库知识点将在下文中讨论）。笔者面试 腾讯 看点部门二面被问到过。

图

图的问题就我个人面试从来没遇到过，不过据我某位哥哥所说，他在进 三星 电子之前有一道面试题就是 深度优先和广度优先问题。

其他的一些 算法

如 A*寻路、霍夫曼编码也偶尔会在某一个领域的公司的面试中被问到，如宝开（《植物大战僵尸》的母公司， 在上海人民广场附近有分公司）。

多线程开发基础

2021年4月8日 23:34

现如今的多核CPU早已经是司空见惯，而多线程编程早已经是“飞入寻常百姓家”。对于大多数桌面应用（与 Web 开发相对），尤其是像后台开发这样的岗位，且面试者是社会人员（有一定的工作经验），如果面试者不熟悉多线程编程，那么一般会被直接 pass 掉。

这里说的“熟悉多线程编程”到底熟悉到什么程度呢？一般包括：知道何种场合下需要新建新的线程、线程如何创建和等待、线程与进程的关系、线程局部存储（TLS 或者叫 thread local）、多线程访问资源产生竞态的原因和解决方案等等、熟练使用所在操作系统平台提供的线程同步的各种原语。

对于 C++ 开发者，你需要：

对于 Windows 开发者，你需要熟练使用 Interlock系列函数、CriticalSection、Event、Mutex、Semphore等API 函数和两个重要的函数 WaitForSingleObject、WaitForMultipleObjects。
对于 Linux 开发者，你需要熟练使用 mutex、semphore、condition_variable、read-write-lock 等操作系统API。
可以使用 C++ 实现一个简单的线程池，当然支持优先级、动态创建线程功能就更好了。

数据库

2021年4月8日 23:34

数据库知识一般在大的互联网企业对应届生不做硬性要求，对于小的互联网企业或社会人士一般有一定的要求。其要求一般包括：

（1）熟悉基本 SQL 操作
包括增删改查（insert、delete、update、select语句）， 排序 order，条件查询（where子语句），限制查询结果数量（LIMIT语句）等

（2）稍微高级一点的 SQL 操作（如 Group by，in，join，left join，多表联合查询，别名的使用，select 子语句等）

（3）索引的概念、索引的原理、索引的创建技巧

（4）数据库本身的操作，建库建表，数据的导入导出

（5）数据库用户权限控制（权限机制）

（6）MySQL的两种数据库引擎的区别

（7）SQL 优化技巧

以上属于对开发的基本的数据库知识要求，你可以找一本相关入门级的数据库图书学习即可。

高级开发除了以上要求还要熟悉 高可用 MySQL 、 主从同步 、 读写分离 、 分表分库 等技术，这些技术的细节一定要清楚，它们是你成为技术专家或者高级架构的必备知识。我们在实际面试时，在讨论高可用服务服务方案时，很多面试者也会和我们讨论到这些技术，但是不少面试者只知道这些技术的大致思想，细节往往说不清楚，细节不会就意味着你的高可用方案无法落地，企业需要可以落地的方案。
这些技术我首推 《高性能 MySQL》这本书，这本书高级开发者一定要通读的 ，另外还有 2 本非常好的图书也推荐一下：一本是《 MySQL 排错指南 》，读完这本书以后，你会对整个“数据库世界”充满了清晰的认识；另外一本是《 数据库索引设计与优化 》，这本书读起来非常舒服，尤其是对于喜欢 算法 和数据结构的同学来说。

网络编程

2021年4月8日 23:35

网络编程这一块，对于应届生或者初级岗位一般只会问一些基础网络通信原理（如三次握手和四次挥手）的socket 基础 API 的使用， 客户端 与服务器端网络通信的流程（回答 【 客户端 创建socket -> 连接server ->收发数据；服务器端创建socket -> 绑定ip和端口号 -> 启动侦听 ->接受 客户端 连接 ->与 客户端 通信收发数据】即可）、TCP 与 UDP的区别等等。

对于工作经验三年以内的社会人士或者一些中级面试者一般会问一些稍微重难点问题，如 select 函数的用法，非阻塞 connect 函数的写法，epoll 的水平和边缘模式、阻塞socket与非阻塞socket的区别、send/recv函数的返回值情形、reuse_addr选项等等。Windows 平台可能还会问 WSAEventSelect 和 WSAAsyncSelect 函数的用法、完成端口（IOCP模型）。

对于三年以上尤其是“号称”自己设计过服务器、看过开源网络通信库代码的面试者，面试官一般会深入问一些问题，这类问题要么是实际 项目 中常见的难题或者网络通信细节，根据我的经验，一般有这样一些问题：

nagle 算法；
keep alive选项；
Linger选项；
对于某一端出现大量CLOSE_WAIT 或者 TIME_WAIT如何解决；
通讯协议如何设计或如何解决数据包的粘包与分片问题；
心跳机制如何设计；（可能不会直接问问题本身，如问如何检查死链）
断线重连机制如何设计；
对 IO Multiplexing 技术的理解；
收发数据包正确的方式，收发缓冲区如何设计；
优雅关闭；
定时器如何设计；
epoll 的实现原理。
举两个例子，让读者感受一下：

笔者曾去 BiliBili 被问过这样一个问题：如果 A 机器与 B 机器网络 connect 成功后从未互发过数据，此时其中一机器突然断电，则另外一台机器与断电的机器之间的网络连接处于哪种状态？

内存数据库&缓存技术

2021年4月8日 23:35

时下以 NoSql key-value 为思想的内存数据库大行其道，广泛地用于各种后台 项目 开发。所以 熟悉一种或几种内存数据库程序已经是面试后台开发的基本要求，而这当中以 Redis 为最典型代表，这里以 redis 为例。

第一层面一般是对 Redis 的基础用法的考察
如考察 Redis 支持的基础数据类型、Redis的数据持久化、事务等。
第二层面不仅考察 Redis 的基础用法，还会深入到 Redis 源码 层面上，如 Redis 的网络通信模型、Redis 各种数据结构的实现等等。
Redis高可用、cluster、哨兵策略等。
笔者以为，无论是从找工作应付面试还是从提高技术的角度，Redis 是一个非常值得学习的开源软件，希望广大读者有意识地去了解、学习它。

另外一些像分布式、RPC、MQ 等技术和 C++ 本身关系不是很紧密，这里就不罗列了。

网络配置

2022年12月10日 16:08

18.0版本后使用/etc/netplan配置网络了

[(43条消息) Ubuntu设置静态IP地址的几种方法_f大熊的博客-CSDN博客_ubuntu配置静态ip地址]

```
# This is the network config written by 'subiquity'
network:
  ethernets:
    ens33:
      dhcp4: false
      addresses: [192.168.2.2/24]
      gateway4: 192.168.2.1
  version: 2
~
~
~
~
~
~
~
~
~
~
```

SSH服务器root登录不了的情况

2022年12月10日 17:20

注意sshd.servcie启动了才能用ssh登录该服务器

vi /etc/ssh/sshd_config

注释掉这行并且这项为yes才允许root登录

```
#LoginGraceTime 2m
PermitRootLogin yes
#StrictModes yes
#MaxAuthTries 6
```

telnet、netcat使用

2023年11月3日 10:01

这两种工具使用TCP协议，本质上是通过网络连接，与另一个主机通信，但是它们会实现特定的功能。从这个点来看，我们可以将这个工具当做一个“客户端”，它可以连接服务器，发送和接受信息，用来测试服务器程序（因为比如telnet默认使用端口23，链接服务器，但服务器只要能接受连接和发送信息即可，至于内容是什么，并不重要，也就是服务器完全可以不遵循telnet协议。）——读smallchat（smallchat/smallchat.c at main • antirez/smallchat (github.com)）代码有感。2023.11.3.

telnet	Telnet（Telecommunication Network）是一种用于远程登录到计算机系统的网络协议，它允许用户通过网络连接到远程计算机，并在远程计算机上执行命令。Telnet协议通常使用TCP/IP协议栈来传输数据。 以下是Telnet协议的一些关键特点和概念： 1. 远程登录：Telnet允许用户从一个计算机系统连接到另一个计算机系统，就像在本地终端上一样执行命令。这对于管理和维护远程计算机非常有用，尤其是在互联网的早期阶段。 2. 明文传输：Telnet在其基本形式下使用明文传输，这意味着用户的命令和数据在网络上以纯文本形式传输，没有加密保护。因此，Telnet在不受信任的网络上使用时存在安全风险，因为攻击者可以拦截和查看传输的数据，包括用户名和密码。 3. 端口号：Telnet默认使用TCP端口23。用户可以使用Telnet客户端连接到远程主机的Telnet服务器。例如，可以在命令行中运行"telnet remote_host"，其中"remote_host"是远程计算机的主机名或IP地址。 4. 安全问题：由于Telnet的明文传输性质，通常不建议在公共网络上使用它，特别是用于敏感信息的传输。为了提高安全性，现代网络中更常用SSH（Secure Shell）协议，它提供了加密和身份验证，以保护远程登录会话中的数据。 总的来说，Telnet是一个古老的协议，虽然在一些特定的用例中仍然有用，但由于其安全性不足，现代网络环境中更常用更安全的替代协议，如SSH。
netcat	Netcat，通常简称为`nc`，是一种网络工具，而不是一种协议。它允许在计算机之间进行网络连接和数据传输。Netcat可以在各种操作系统上运行，包括Linux、Windows和macOS。虽然Netcat本身不是一个协议，但它是一个强大的工具，用于创建各种网络连接和执行多种网络任务。 Netcat的功能包括：

	1. 端口扫描：Netcat可用于扫描远程主机的端口，以确定哪些端口处于开放状态，这对于安全审计和网络故障排除很有用。 2. 网络监听：Netcat可以用作服务器，监听特定端口并接受传入的连接。这可用于创建简单的网络服务或测试网络应用程序。 3. 数据传输：Netcat允许在计算机之间传输数据。这可用于在不同主机之间传输文件、发送文本数据或执行其他网络通信任务。 4. 反向Shell：Netcat可以用于建立反向shell连接，这对于远程管理远程主机非常有用。 5. 端口转发：Netcat可以用于设置端口转发，将数据从一个端口转发到另一个端口，这对于网络中继和代理很有用。 虽然Netcat是一个功能强大的工具，但它也可以被滥用，因此在使用时应格外小心。在一些情况下，恶意用户可能会使用Netcat来建立不安全的连接或进行攻击。因此，要确保在合法和合适的情况下使用Netcat，并遵守网络安全最佳实践。 Netcat本身并不是一种网络协议，它是一个网络工具，用于处理各种网络任务。

gedit--可视化文本编辑器

2020年11月16日 0:59

Unix Shells：bash、fish、ksh、tcsh、zsh

2020年11月30日 15:44

参考资料： https://hyperpolyglot.org/unix-shells

Grammar

	bash	fish	ksh	tcsh	zsh
simple command	`ls`	`ls`	`ls`	`ls`	`ls`
simple command with argument	`echo hi`	`echo hi`	`echo hi`	`echo hi`	`echo hi`
simple command with redirect	`ls > /tmp/ls.out`	`ls > /tmp/ls.out`	`ls > /tmp/ls.out`	`ls > /tmp/ls.out`	`ls > /tmp/ls.out`
simple command with environment variable	`EDITOR=vi git commit`	`env EDITOR=vi git commit`	`EDITOR=vi git commit`	`env EDITOR=vi git commit`	`EDITOR=vi git commit`
pipeline	`ls | wc`	`ls | wc`	`ls | wc`	`ls | wc`	`ls | wc`
sublist separators	`&& ||`	*none*	`&& ||`	`&& ||`	`&& ||`
list terminators	`; &`	`; &`	`; &`	`; &`	`; &`
group command	`{ ls; ls;} | wc`	`begin; ls; ls; end | wc`	`{ ls; ls;} | wc`	*none*	`{ ls; ls;} | wc`
subshell	`(ls; ls) | wc`	`fish -c 'ls; ls' | wc`	`(ls; ls) | wc`	`(ls; ls) | wc`	`(ls; ls) | wc`

Shells read input up to an unquoted newline and then execute it. An unquoted backslash followed by a newline are discarded and cause the shell to wait for more input. The backslash and newline are discarded before the shell tokenizes the string, so long lines can be split anywhere outside of single quotes, even in the middle of command names and variable names.

In the shell grammar, *lists* contain *sublists*, which contain *pipelines*, which contain *simple commands*.

Subshells and *grouping* can be used to put a list in a pipeline. Subshells and groups can have newlines, but the shell defers execution until the end of the subshell or group is reached.

The section on execution control describes structures which do not fit into the simple grammar and execution model outlined here. The shell will not execute any of the control structures until the end keyword is reached. As a result, the control structure can contain multiple statements separated by newlines. Execution control structures cannot be put into pipelines.

创建密钥

2023年1月17日 9:58

ssh-keygen -t rsa

SSH建立信任互传

2022年12月29日 17:13

建立信任互传	ssh-copy-id -p7777 root@ip

ssh-copy-id命令的介绍，使免密更方便

ssh-copy-id命令可以把本地的ssh公钥文件安装到远程主机对应的账户下。

达到的功能:
ssh-copy-id - 将你的公共密钥填充到一个远程机器上的authorized_keys文件中。

描述:

```
**ssh-copy-id 是一个实用ssh去登陆到远程服务器的脚本（假设使用一个登陆密码，
因此，密码认证应该被激活直到你已经清理了做了多个身份的使用）。
它也能够改变远程用户名的权限，~/.ssh和~/.ssh/authorized_keys
删除群组写的权限（在其它方面，如果远程机上的sshd在它的配置
文件中是严格模式的话，这能够阻止你登陆。）。

如果这个 “-i”选项已经给出了，然后这个认证文件（默认是~/.ssh
/id_rsa.pub）被使用，不管在你的ssh-agent那里是否有任何密钥。
另外，命令 “ssh-add -L” 提供任何输出，它使用这个输出优先于
身份认证文件。如果给出了参数“-i”选项，或者ssh-add不产生输出，
然后它使用身份认证文件的内容。一旦它有一个或者多个指纹，它使
用ssh将这些指纹填充到远程机~/.ssh/authorized_keys文件中。**
```

fio

2022年12月29日 17:14

filename参数指定某个要测试的裸设备（硬盘或分区），切勿在系统分区做测试，会破坏系统分区，而导致系统崩溃。若一定要测试系统分区较为安全的方法是：在根目录下创建一个空目录，在测试命令中使用directory参数指定该目录，而不使用filename参数。

[fio 命令入门到跑路（千万不能在系统所在的分区测试硬盘性能）- 程序员大本营 (pianshen.com)]

未命名

2023年2月15日 11:02

linux抓包工具有哪些

原创 2022-04-29 18:15:19 9748

linux抓包工具：1、Charles，通过成为电脑或者浏览器的代理，截取请求和请求结果达到分析抓包的目的；2、Wireshark，一款跨平台的抓包工具；3、tcpdump，一个网络数据采集分析工具，可抓取数据并分析；4、Netstat等。

来自 <https://www.php.cn/linux-491527.html>

Valgrind

2023年3月23日 15:41

Valgrind 是一款流行的开源工具集，用于在 Linux 环境下进行程序性能分析、内存调试和代码优化等任务。它支持多种语言，包括 C、C++、Objective-C 和 Ada，并提供了许多有用的功能，如内存泄漏检测、堆栈跟踪、进程间通信分析等。Valgrind 的核心是一个称为“仪器”的动态二进制重写引擎，在运行时对程序进行修改，从而收集有关其执行情况的详细信息。 Valgrind 可以帮助开发人员快速定位并解决各种编程错误和内存问题，提高代码质量和系统性能。

来自 <http://newb.site/chatgpt/#/chat/1679556874609>

使用 Valgrind 可以分为以下几个步骤:

1. 安装 Valgrind：在终端中输入适合您的 Linux 发行版的命令，如 sudo apt-get install valgrind。
2. 编译程序时加入 -g 选项：这样可生成可调试的二进制文件。
3. 运行程序时使用 Valgrind：在终端中输入 valgrind --tool=工具名 [选项] 程序名 即可运行程序并使用 Valgrind 工具进行分析。例如，可以使用以下命令运行程序并检查内存泄漏：

   复制代码
   ```
   valgrind --leak-check=yes ./my_program
   ```

4. 分析分析结果：Valgrind 在运行程序时会输出许多关于程序执行情况的信息，包括内存泄漏、读写越界等错误。开发人员可以根据这些信息快速定位和修复问题。

   需要注意的是，使用 Valgrind 对程序进行分析可能会使程序运行缓慢，并且在某些情况下可能会导致不准确的结果。因此，建议在测试环境下使用 Valgrind 进行分析，而不是在生产环境中使用。

来自 <http://newb.site/chatgpt/#/chat/1679556874609>

mdadm

2023年4月26日 11:25

管理Linux系统阵列。

例子	[root@localhost bin]# mdadm --create /dev/md1 --chunk=64 --level=1 --raid-devices=2 --run /dev/nvme0n1 /dev/nvme1n1
	[root@localhost ~]# mdadm --misc --detail /dev/md1 /dev/md1: Version : 1.2 Creation Time : Wed Apr 26 16:03:54 2023 Raid Level : raid1 Array Size : 1171981888 (1117.69 GiB 1200.11 GB) Used Dev Size : 1171981888 (1117.69 GiB 1200.11 GB) Raid Devices : 2 Total Devices : 2 Persistence : Superblock is persistent Intent Bitmap : Internal Update Time : Wed Apr 26 16:03:54 2023 State : clean Active Devices : 2 Working Devices : 2 Failed Devices : 0 Spare Devices : 0 Consistency Policy : bitmap Name : localhost:1 (local to host localhost) UUID : f96980d8:f334f5bb:c53a3f3d:056a6a85 Events : 0 Number Major Minor RaidDevice State 0 259 0 0 active sync /dev/nvme0n1 1 259 1 1 active sync /dev/nvme1n1

未命名

2023年1月18日 10:03

Linux依赖包搜索

2021年12月20日
14:36

https://pkgs.org/

https://rpmfind.net/linux/rpm2html/

未命名

2023年1月18日 10:04

likely和unlikely函数的含义

2021年12月29日
11:42

总之,likely与unlikely互换或不用都不会影响程序的正确性。但可能会影响程序的效率。

if(likely(foo)) //认为foo通常为1
if(unlikely(foo)) //认为foo通常为0

来自 <https://blog.csdn.net/punk_lover/article/details/45916773>

container_of

2023年1月18日 10:17

0指针

2023年1月18日 10:17

systemctl服务

2023年2月13日 11:33

service servcie_name start

[Unit]
Description=service_name service
After=A
Before=B
启动时，在B之前执行，A之后执行，关机时，相反

Linux kernel-0.11

2023年2月21日 11:35

https://github.com/sunym1993/flash-linux0.11-talk